

软件工程

Software Engineering

MS Roslyn

2026 年 10 月 9 日

摘要

本教程系统讲解软件工程的核心方法与建模技术：从软件过程模型切入，涵盖需求工程、结构化分析与设计、面向对象与 UML 建模、数据建模与 ER、设计原则与模式、软件测试，以及项目管理与质量度量。教程强调「过程 + 建模 + 质量」三条主线，把抽象的方法论落到可操作的图表与度量上，帮助读者掌握从需求到交付的完整工程实践。

目录

第一章	软件工程概述与过程模型	6
1.1	软件、软件危机与软件工程	7
1.1.1	软件的定义与本质	7
1.1.2	软件危机	7
1.1.3	软件工程的定义与目标	7
1.2	软件过程模型	8
1.2.1	瀑布模型	8
1.2.2	迭代增量模型	8
1.2.3	螺旋模型	9
1.2.4	原型模型	9
1.2.5	敏捷模型	9
1.2.6	过程模型对比	10
1.3	软件生命周期、阶段与缺陷代价	10
1.3.1	生命周期各阶段与交付物	10
1.3.2	缺陷修复代价随阶段增长	11
1.4	敏捷宣言、Scrum 与 XP	12
1.4.1	敏捷宣言	12
1.4.2	Scrum: 角色、工件与仪式	12
1.4.3	XP 核心实践	12
1.5	软件质量与度量	13
1.5.1	ISO 质量模型要点	13
1.5.2	常用度量	13
1.6	本章小结与常见误区	14
第二章	需求工程	15
2.1	需求与需求工程	16
2.1.1	什么是需求	16
2.1.2	需求工程的五项活动	16
2.2	需求获取方法	17
2.3	需求分类	17
2.3.1	功能需求与非功能需求	17
2.3.2	非功能需求的类别	18
2.4	软件需求规格说明	18
2.4.1	规格说明的特征	18

2.4.2	SRS 的文档结构	19
2.5	用例建模	19
2.5.1	参与者、用例与关系	19
2.5.2	用例图	20
2.5.3	用例描述模板	20
2.5.4	用例描述示例	21
2.6	需求管理	22
2.6.1	基线与变更控制	22
2.6.2	需求追溯矩阵	22
2.7	本章小结	23
2.8	常见误区	23
第三章	结构化分析与设计	24
3.1	结构化分析思想	25
3.1.1	为什么需要结构化	25
3.1.2	三条核心原则	25
3.2	数据流图 (DFD)	26
3.2.1	四种基本元素	26
3.2.2	0 层与 1 层分解	26
3.2.3	父子图平衡原则	27
3.2.4	一个完整的小例子	27
3.3	数据字典	27
3.3.1	记号的语义	27
3.3.2	一个完整条目示例	28
3.4	结构化设计：模块、耦合与内聚	29
3.4.1	模块结构图的量与术语	29
3.4.2	耦合 (Coupling)	29
3.4.3	内聚 (Cohesion)	30
3.4.4	高内聚、低耦合	30
3.5	变换分析与事务分析	30
3.5.1	变换分析 (Transform Analysis)	30
3.5.2	事务分析 (Transaction Analysis)	31
3.6	本章小结与常见误区	31
3.6.1	本章小结	31
3.6.2	常见误区	31
第四章	面向对象与 UML	32
4.1	类图	33
4.1.1	类与成员	33
4.1.2	类之间的关系	34
4.1.3	多重性	34
4.1.4	类图示例	35
4.2	用例图	36

4.2.1	要素与关系	36
4.3	时序图	37
4.3.1	要素	37
4.4	状态图与活动图	38
4.4.1	状态图	38
4.4.2	活动图	39
第五章	数据建模与 ER	40
5.1	数据建模概述	41
5.2	实体、属性与键	42
5.2.1	实体与实体集	42
5.2.2	属性的分类	42
5.2.3	键	43
5.2.4	弱实体	43
5.3	联系	44
5.3.1	联系的度	44
5.3.2	基数	44
5.3.3	参与约束	44
5.3.4	属性化联系	45
5.4	ER 到关系模式的转换	45
5.4.1	一对一与一对多	46
5.4.2	多对多	46
5.4.3	弱实体、多值属性与三元联系	47
5.4.4	转换实例	47
5.5	函数依赖与范式	47
5.5.1	函数依赖	47
5.5.2	范式的定义	48
5.5.3	从非规范化到 3NF：一个完整分解	48
5.6	本章小结	49
5.7	常见误区	49
第六章	设计原则与模式	51
6.1	设计过程与目标	52
6.1.1	从需求到实现	52
6.1.2	四条基本目标	52
6.2	软件设计原则	53
6.2.1	单一职责原则（SRP）	53
6.2.2	开闭原则（OCP）	53
6.2.3	里氏替换原则（LSP）	54
6.2.4	接口隔离原则（ISP）	54
6.2.5	依赖倒置原则（DIP）	54
6.3	设计模式	55
6.3.1	模式分类	55

6.3.2	代表模式	55
6.4	软件体系结构风格	56
6.5	设计度量	57
6.6	本章小结与常见误区	58
第七章	软件测试	59
7.1	测试目标与原则	60
7.1.1	测试的定义与目标	60
7.1.2	测试与调试的区别	60
7.1.3	测试的基本原则	60
7.2	测试层次与 V 模型	61
7.2.1	四个测试层次	61
7.2.2	V 模型	62
7.2.3	集成测试策略、桩与驱动	62
7.3	黑盒测试方法	63
7.3.1	等价类划分	63
7.3.2	边界值分析	64
7.3.3	因果图与判定表	64
7.3.4	错误推测	64
7.4	白盒测试与逻辑覆盖	65
7.4.1	被测程序示例	65
7.4.2	六种覆盖准则	65
7.4.3	基本路径测试	66
7.5	测试用例设计综合与回归测试	66
7.5.1	测试用例的要素与设计顺序	66
7.5.2	回归测试	67
7.6	本章小结	67
第八章	项目管理与质量	68
8.1	规模与成本估算	69
8.1.1	代码行 LOC 与功能点 FP	69
8.1.2	COCOMO 模型	69
8.2	进度计划	70
8.2.1	甘特图	70
8.2.2	PERT 与关键路径法 CPM	70
8.3	风险管理	71
8.3.1	风险识别与评估	71
8.3.2	风险应对策略	72
8.4	软件质量	72
8.4.1	ISO 质量模型	72
8.4.2	质量度量	73

第一章 软件工程概述与过程模型

软件工程用「工程化」的方法控制软件的成本、进度与质量。本章建立「软件危机-过程模型-生命周期-敏捷」四块基石，并引入缺陷代价与质量度量的定量视角。

1.1 软件、软件危机与软件工程

1.1.1 软件的定义与本质

软件是计算机程序、数据以及相关文档的集合。程序是「做什么、怎么做」的可执行描述；数据是程序处理的对象与配置；文档记录需求、设计、接口与使用说明，是软件可被理解、维护与交接的载体。三者缺一不可：只有程序而无文档，软件便无法维护；只有文档而无程序，软件便无法运行。

表 1.1: 软件与硬件特性的对比

维度	硬件	软件
本质	物理实体	逻辑产品，无形
制造方式	传统意义的「制造」	一次开发，成本集中在设计
磨损	物理磨损、老化	无磨损，但会因变更而「退化」
失效曲线	浴盆曲线（早期/偶发/耗损）	理想为阶梯退化，实际因维护引入缺陷而上升
复制成本	随产量分摊	近乎零边际成本
度量	可直接测量几何、重量	多为间接度量（复杂度、缺陷密度）

软件不会像硬件那样「用坏」，但会因持续的变更而逐渐腐化：每一次修改都可能在原有结构中埋下不一致，最终使命名、接口与文档彼此脱节，这一现象称为**软件退化或腐化**。因此维护不是可选项，而是与开发同等重要的工程活动。

1.1.2 软件危机

20 世纪 60 年代，硬件能力迅速提升、软件规模急剧膨胀，但开发仍停留在「个人手工作坊」模式，导致大量项目失败。这种「软件的生产与需求之间的严重不适应」被称为**软件危机**。

表 1.2: 软件危机的典型表现

表现	说明
需求不清且频繁变更	用户说不清需求，开发前未充分确认，返工频繁
进度与成本失控	估算靠猜，项目频繁延期、超预算
质量低下	缺陷多、可靠性差、性能不达标
维护困难	结构混乱、文档缺失，修改一处影响一片
生产率增长缓慢	硬件按摩尔定律增长，软件生产率提升有限

软件危机的根源不在技术工具，而在于**缺乏工程化的组织与管理**：把软件当作「编程」而非「工程」。软件工程正是对这一危机的系统性回应。

1.1.3 软件工程的定义与目标

软件工程是「将系统化、规范化、可度量的方法应用于软件的开发、运行和维护，即把工程方法运用于软件」。其目标是：在**给定成本与进度**下，交付**满足需求、质量可用**的软件，并使软件在其整个生命周期内可维护、可演化。

软件工程区别于「会编程」的关键在于三个方面：**过程**（有组织的开发步骤与模型）、**方法**（需求、设计、测试等具体技术）、**工具**（支撑过程与方法的自动化手段）。三者常被称为软件工程的「三要素」。

1.2 软件过程模型

软件过程是开发软件时一组有序的活动、约束与产物。**过程模型**（又称生命周期模型）是对过程的抽象，规定「先做什么、再做什么、如何迭代、何时交付」。没有放之四海皆准的模型，模型的选择取决于需求稳定性、项目规模、风险水平与团队能力。

1.2.1 瀑布模型

瀑布模型把开发划分为若干严格顺序的阶段：需求分析、设计、编码、测试、维护。上一阶段的产物经评审通过后才能进入下一阶段，阶段之间以文档为接口。

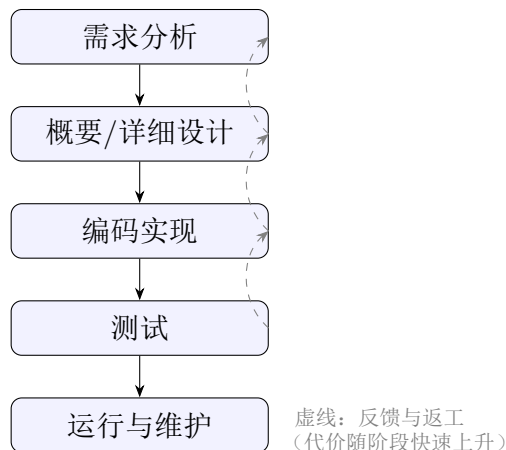


图 1.1: 瀑布模型：阶段线性推进，反馈导致大量返工

机制：文档驱动、阶段性评审、一次性交付。**优点：**过程清晰、里程碑明确、便于管理与合同外包。**缺点：**难以应对需求变更；缺陷发现晚，返工代价高；用户到后期才见到可运行系统。**适用：**需求稳定、技术成熟、以合同为约束的工程项目。

1.2.2 迭代增量模型

迭代增量模型把系统切分为若干**增量**，每一轮**迭代**完成一个可运行、可交付的子集，逐步逼近完整系统。它承认需求会演进，通过多轮循环来收敛。

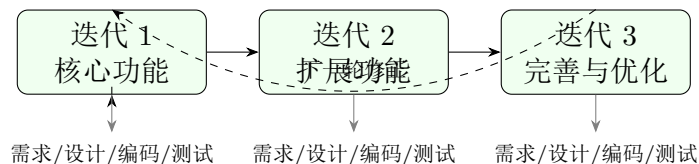


图 1.2: 迭代增量模型：每轮迭代交付一个可运行的增量

机制：把大系统拆小，每轮都走一遍「需求-设计-实现-测试」，尽快交付可用增量以获取反馈。**优点：**早期见效、风险分散、适应变更。**缺点：**需要良好的架构与集成策略，否则后期集成困难；迭代次数与范围难控。**适用：**中大型、需求渐进明确的项目。

1.2.3 螺旋模型

螺旋模型由 Boehm 提出，把迭代与**风险分析**结合。每圈螺旋包含四个象限：制定计划、风险分析、实施工程、客户评估。

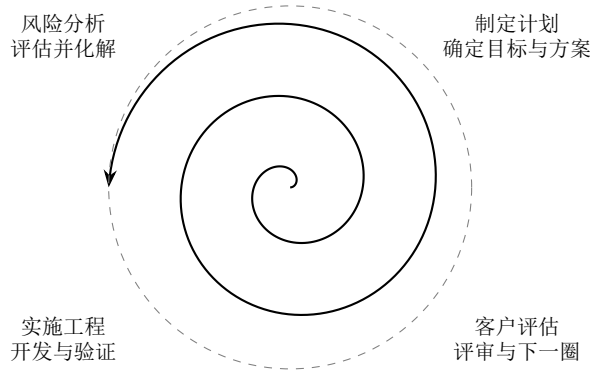


图 1.3: 螺旋模型：每一圈以风险分析为核心，半径随进度增大

机制：每圈先识别并评估风险，风险高的部分优先探索与原型验证。**优点：**显式管理风险，适合高风险项目。**缺点：**风险分析需要专门技能，管理成本高，小项目不划算。**适用：**大型、高风险、需求不确定的项目。

1.2.4 原型模型

原型模型先快速构建一个**可运行原型**，用它澄清模糊需求，再据此开发正式产品。原型可分为**探索型**（需求不明确时用于沟通）与**演化型**（逐步演化成最终产品）。

机制：快速构建、用户试用、反馈修正。**优点：**有效澄清需求、降低沟通误差。**缺点：**用户易把原型误当成品；抛弃型原型若不舍弃会损害质量；工具与快速开发要求高。**适用：**需求模糊、用户参与度高、界面与交互密集的项目。

1.2.5 敏捷模型

敏捷以「短迭代、持续交付、响应变化」为核心，强调可工作软件高于详尽文档、客户合作高于合同谈判。它并非单一模型，而是一组价值观下的实践族（Scrum、XP、看板等）。

机制：2-4 周短迭代内完成一小批高价值功能，持续集成与持续反馈。**优点：**适应变化、交付频繁、贴近用户。**缺点：**大规模协同与文档、合规场景挑战大；对团队纪律与用户参与要求高。**适用：**需求多变、团队小型且自组织、可接受渐进交付的项目。

1.2.6 过程模型对比

表 1.3: 常见过程模型对比

模型	核心机制	优点	缺点 / 适用
瀑布	线性阶段、文档驱动	清晰、里程碑明确、易管理	难应变、返工贵 / 需求稳定、合同型
迭代增量	多轮交付可运行增量	早期见效、适应变更	集成难、范围难控 / 中大型、渐进需求
螺旋	迭代 + 显式风险分析	主动控风险	成本高、需专门技能 / 高风险大型项目
原型敏捷	先做可运行原型 短迭代、响应变化	澄清需求、沟通好 灵活、交付频繁	易被误当成品 / 需求模糊 重协同、文档弱 / 需求多变、小团队

1.3 软件生命周期、阶段与缺陷代价

1.3.1 生命周期各阶段与交付物

软件生命周期是从需求提出到软件退役的全过程。虽然模型不同，但其中反复出现的核心活动与交付物基本一致。

表 1.4: 生命周期各阶段的主要活动与交付物

阶段	主要活动	关键交付物	质量关口
需求	获取、分析、确认需求	需求规格说明书、用例模型	需求评审通过
设计	架构与详细设计	设计说明书、类图/结构图、接口定义	设计评审通过
实现	编码、单元测试	源码、单元测试用例、构建脚本	代码评审、CI 通过
测试	集成、系统、验收测试	测试计划、测试报告、缺陷记录	质量门禁通过
维护	改正、适应、完善、预防	变更记录、回归测试、发布说明	回归无退化
退役	数据迁移、下线	迁移方案、退役公告	数据完整迁移

1.3.2 缺陷修复代价随阶段增长

一个缺陷发现得越晚，修复代价越高：需求阶段只需改一句话，维护阶段可能要重做设计、修改代码、重跑全部测试。经验上，修复代价随阶段序号近似**指数增长**：

$$C_{\text{fix}}(s) \propto e^{k \cdot s}, \quad s = 1, 2, \dots \text{ 为阶段序号} \tag{1.1}$$

其中 k 为增长系数。 k 越大，说明该组织改动成本越高，越应把质量「前置」。这一规律是**尽早测试、持续测试与缺陷左移**的根本依据。

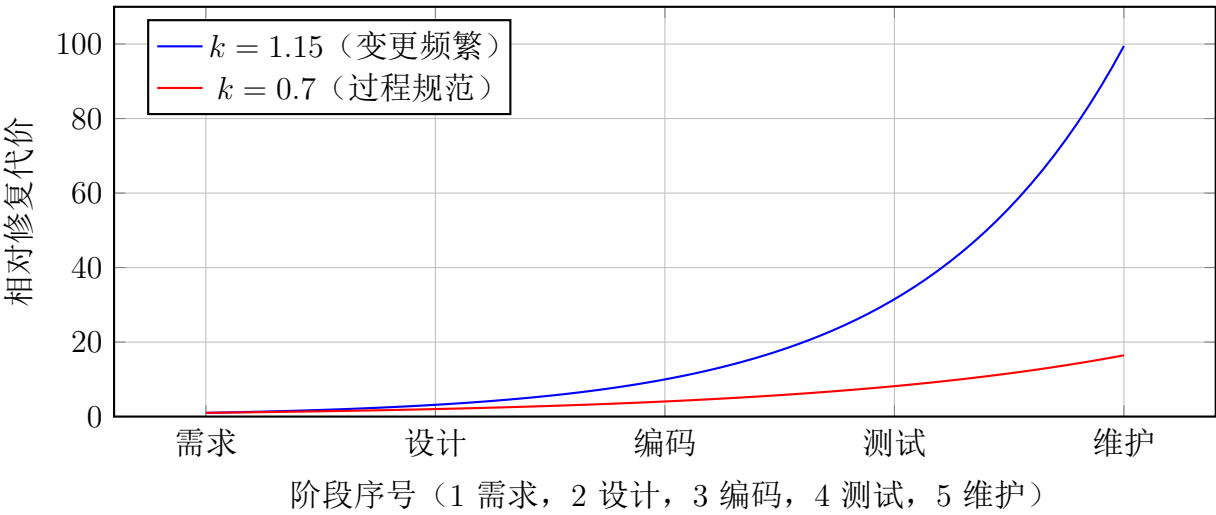


图 1.4: 缺陷修复代价随阶段指数增长 (k 越大曲线越陡)

表 1.5: 「缺陷左移」的直观示例：同一缺陷在不同阶段修复 ($k = 1.15$ ，需求阶段代价记为 1)

修复阶段	阶段序号 s	相对代价 $e^{k(s-1)}$	与需求阶段相比
需求	1	1.0	1×
设计	2	3.2	≈ 3×
编码	3	10.0	≈ 10×
测试	4	31.5	≈ 30×
维护	5	99.5	≈ 100×

例 1 (缺陷代价估算) 某团队在需求阶段修复一个缺陷成本为 500 元，增长系数 $k = 1.15$ 。若该缺陷直到维护阶段 ($s = 5$) 才被发现，估算其修复代价；并说明缺陷左移到设计阶段可节省多少。

解：由式 (1)，维护阶段相对代价 $e^{1.15 \times (5-1)} = e^{4.6} \approx 99.5$ ，故

$$C_{\text{maint}} \approx 500 \times 99.5 = 49,750 \text{ 元.}$$

若在设计阶段 ($s = 2$) 发现并修复，相对代价 $e^{1.15} \approx 3.16$ ，代价约 $500 \times 3.16 \approx 1,580$ 元。节省约 $49,750 - 1,580 \approx 48,170$ 元，即约 97%。结论：需求与设计评审每投入少量成本，都可能有数十倍的回报。

1.4 敏捷宣言、Scrum 与 XP

1.4.1 敏捷宣言

2001 年发布的《敏捷宣言》提出四条价值观：**个体与互动**高于**流程与工具**；**可工作的软件**高于**详尽的文档**；**客户合作**高于**合同谈判**；**响应变化**高于**遵循计划**。宣言同时承认右项也有价值，只是更看重左项，速度与灵活性由此而来。

1.4.2 Scrum：角色、工件与仪式

Scrum 是最流行的敏捷框架，通过固定的角色、工件与仪式保证「短周期、可预测、可改进」。

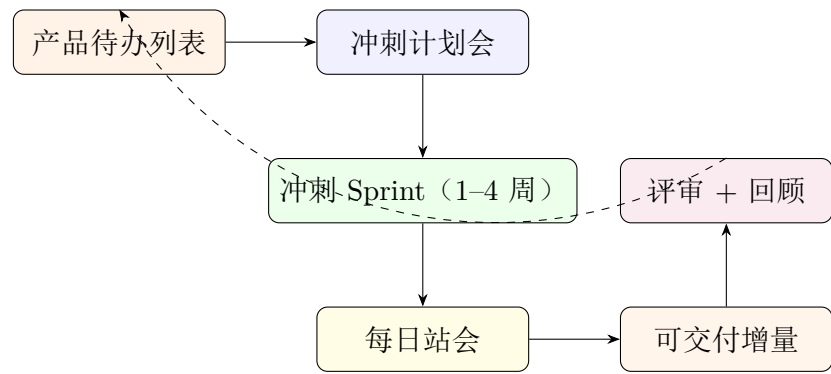


图 1.5: Scrum 冲刺流程：从产品待办列表到增量，再经评审与回顾进入下一轮

表 1.6: Scrum 的角色、工件与仪式

类别	组成	职责 / 说明
角色	产品负责人 (PO)	管理产品待办列表，对价值与优先级负责
角色	Scrum Master	保障流程、移除障碍、促进改进
角色	开发团队	自组织地交付增量
工件	产品待办列表	全部需求的有序列表 (PO 维护)
工件	冲刺待办列表	本轮选定的任务与计划
工件	增量	满足「完成定义」的可工作产品
仪式	冲刺计划会	选定本轮目标与待办
仪式	每日站会	同步进展、暴露障碍 (15 分钟)
仪式	冲刺评审	向干系人演示增量并收集反馈
仪式	冲刺回顾	改进团队工作方式

1.4.3 XP 核心实践

极限编程 (XP) 强调工程实践的极致执行，核心技术实践包括：

- 结对编程：两人共用一台机器，一人编码、一人审查，实时保证质量与知识传播。
- 测试驱动开发 (TDD)：先写失败的测试（红），再写刚好通过的代码（绿），最后重构（重构）。
- 持续集成：频繁把代码集成到主干并自动构建、测试，尽早暴露集成缺陷。
- 小步发布：以极短周期交付小版本，快速获取真实反馈。
- 简单设计：不做过度设计，只满足当前需求，按需重构。

- **重构**：在不改变外部行为的前提下持续改善内部结构。

1.5 软件质量与度量

1.5.1 ISO 质量模型要点

ISO/IEC 25010 将软件产品质量划分为若干可评价的特性，用于把「好不好」拆解为可度量的维度。

表 1.7: ISO/IEC 25010 产品质量特性要点

质量特性	含义要点
功能适合性 (Functional Suitability)	功能完整、正确，覆盖用户的真实需求
性能效率 (Performance Efficiency)	时间行为、资源利用、容量满足要求
兼容性 (Compatibility)	与其他系统共存、可互操作
易用性 (Usability)	易学、易用、界面友好、可访问
可靠性 (Reliability)	成熟度、可用性、容错、可恢复
安全性 (Security)	保密性、完整性、不可否认、可审计
可维护性 (Maintainability)	模块化、可复用、易分析、易修改、易测试
可移植性 (Portability)	可适应、可安装、可替换

1.5.2 常用度量

缺陷密度把缺陷数按规模归一，便于跨项目比较：

$$D = \frac{N_{\text{defect}}}{\text{Size}} \quad [\text{缺陷/KLOC}], \tag{1.2}$$

其中 N_{defect} 为统计期内发现的缺陷数，Size 为代码规模（千行，KLOC）。

可靠性用失效概率度量。若失效服从速率为 λ 的指数分布，则在时刻 t 仍正常工作的可靠度为

$$R(t) = e^{-\lambda t}, \quad \text{平均无故障时间 MTBF} = \frac{1}{\lambda}. \tag{1.3}$$

由式 (3) 可见： λ 越小， $R(t)$ 越高、MTBF 越长。可靠性目标通常转化为对 λ 或 MTBF 的量化要求。

例 2（缺陷密度与可靠性）某模块共 40 KLOC，集成测试期间发现 120 个缺陷，且上线后运行 1,000 小时发生 2 次失效（按指数分布、失效相互独立）。求缺陷密度，并估计 MTBF 与运行 500 小时的可靠度。

解：缺陷密度

$$D = \frac{120}{40} = 3.0 \text{ 缺陷/KLOC.}$$

失效速率 $\lambda = 2/1000 = 2 \times 10^{-3}$ /小时，故

$$\text{MTBF} = \frac{1}{\lambda} = 500 \text{ 小时}, \quad R(500) = e^{-2 \times 10^{-3} \times 500} = e^{-1} \approx 0.368.$$

即平均无故障运行约 500 小时；运行 500 小时后可靠度约 36.8%，若要求更可靠则须降低 λ （通过修复缺陷、加固设计实现）。

1.6 本章小结与常见误区

本章主线可归纳为四块基石：

- **软件与软件危机**：软件是程序、数据与文档的逻辑产品，无磨损但会腐化；危机源于缺乏工程化组织。
- **过程模型**：瀑布、迭代增量、螺旋、原型、敏捷各有取舍，选择取决于需求稳定性、规模与风险。
- **生命周期与缺陷代价**：各阶段有明确交付物与质量关口；缺陷修复代价随阶段指数增长，故须质量前置。
- **敏捷与质量度量**：敏捷用短迭代快速响应变化；质量可用 ISO 特性、缺陷密度与可靠性等指标量化。

常见误区

- **「敏捷等于不写文档、不设计」**：敏捷是「可工作软件高于详尽文档」，而非取消设计。没有架构与约定，短迭代很快会陷入返工。
- **「瀑布已过时」**：在需求稳定、强合同约束的场景下，瀑布仍高效；问题不在模型本身，而在于用错场合。
- **「测试是编码之后的一个阶段」**：测试应贯穿全过程并尽早介入，缺陷左移的收益远超事后测试。
- **「缺陷密度越低质量越好」不可一概而论**：它受规模口径、缺陷定义与测试充分性影响，须结合场景解读，不能跨口径直接比较。
- **「覆盖率越高越好」**：高覆盖率不等于高质量，关键在于测到业务关键路径与边界条件。

第二章 需求工程

需求工程把「用户想要什么」转化为「可验证、可追溯、可管理的规格」，是决定项目成败的第一道关口。

需求工程 (Requirements Engineering, RE) 是软件生命周期中**最先、也最难逆转**的阶段。大量实证研究表明，需求阶段的缺陷若拖到测试或维护阶段才被发现，其修复代价将呈数量级增长。本章建立一条主线：需求是什么 → 如何获取 → 如何分类与规格化 → 如何用例刻画 → 如何管理变更。

2.1 需求与需求工程

2.1.1 什么是需求

需求 (Requirement) 是对目标系统应当做什么、以及做到什么程度的陈述，是设计、实现、测试与验收的共同基准。需求既包含用户期望，也包含业务、法律、技术与环境施加的约束。需求描述的是问题域 (problem space) 而非解决方案 (solution space) ——这是全文最重要的分界线：

- **问题域 (需求)：**系统要解决什么问题、满足谁的什么目标。
- **解空间 (设计)：**用什么架构、算法、数据库、框架去解决。

干系人 (Stakeholder) 是所有受系统影响或能影响系统的人与组织：最终用户、业务方、运维、法务、监管机构、外部系统接口方等。干系人缺失是需求不完整的最常见根因，故需求工作始于干系人识别。

2.1.2 需求工程五项活动

需求工程不是一次性的「收集」，而是一个**迭代**过程，由五项相互衔接的活动组成：

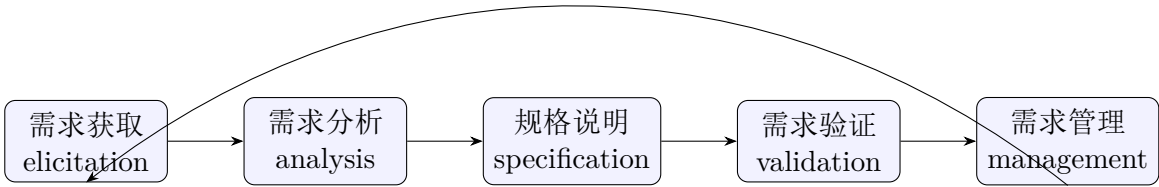


图 2.1: 需求工程五项活动：获取 → 分析 → 规格说明 → 验证 → 管理（管理贯穿始终，并反馈驱动下一轮获取）

表 2.1: 需求工程五项活动的目标与产出

活动	目标	关键手段	主要产出
获取 (Elicitation)	发现并收集原始需求	访谈、问卷、观察、原型、JAD	需求来源清单、原始素材
分析 (Analysis)	甄别、归类、协商、建模	需求分类、冲突协商、用例建模	需求模型、优先级
规格说明 (Specification)	写成规范、可验证的文档	结构化条目、图表、模板	SRS 文档
验证 (Validation)	确认写下的就是用户要的	评审、原型确认、检查表	评审记录、缺陷清单
管理 (Management)	控制变更与追溯	基线、变更控制、追溯矩阵	基线版本、CR 记录、矩阵

常见误解：把「获取」等同于「需求工程」。实际上，捕获到的原始需求往往是冲突、模糊、冗余的，「分析」与「验证」才是把素材打磨成规格的关键。

2.2 需求获取方法

获取方法的选择取决于信息的性质与干系人的可及性。不同方法在覆盖广度、深度、成本与偏误上各有取舍，实践中通常组合使用。

表 2.2: 常用需求获取方法对比

方法	做法	优点	局限与适用
访谈 (Interview)	与干系人一对一或小组交谈	深入挖掘隐含需求、可追问	成本高、依赖技巧；适合关键干系人
问卷 (Questionnaire)	结构化问题批量收集	覆盖广、易统计、成本低	难以追问细节；适合已知问题的普查
观察 (Observation)	到现场观察真实 workflow	发现用户「未说出口」的需求	耗时、可能干扰工作；适合流程型系统
原型 (Prototype)	用可运行或低保真原型试错	澄清模糊需求、反馈直观	易被误当成最终产品；适合界面与交互
文档分析 (Document Analysis)	分析现有表单、手册、旧系统	快速获取业务规则与数据	文档可能过时、未必反映实况
联合应用开发 (JAD)	多方干系人集中研讨会	快速达成共识、暴露冲突	组织成本高、需专业主持；适合跨部门项目

联合应用开发 (Joint Application Development, JAD) 是一种结构化的群体获取技术：把用户、业务方、开发者集中到同一会议室，由**中立主持人 (facilitator)** 引导，在若干天内通过议程、白板与原型快速收敛需求。JAD 的价值在于把**冲突提前暴露在会议桌上**，而不是让它在编码后期爆发。

- **选择原则：**深入单个关键角色用访谈；广撒网用问卷；看「真实行为」用观察；需求模糊用原型；有历史资料用文档分析；需要快速达成跨部门共识用 JAD。
- **三角验证：**同一需求尽量用两种以上方法交叉确认，降低单一来源的偏误。

2.3 需求分类

2.3.1 功能需求与非功能需求

需求首先分为**功能需求 (Functional Requirement)** 与**非功能需求 (Non-functional Requirement)**。前者描述系统「做什么」，后者描述系统「做得怎样」。

表 2.3: 功能需求与非功能需求对比

维度	功能需求	非功能需求
关注点	系统「做什么」	系统「做得怎样」
表述	输入 → 处理 → 输出	可度量的质量指标
示例	用户可下单、支付、退款	响应 < 2s、并发 1000、可用性 99.9%
验证方式	功能测试、验收测试	性能/安全/压力/恢复测试
失败后果	功能缺失或行为错误	系统不可用或不达标

2.3.2 非功能需求的类别

非功能需求常被忽视，却往往是系统成败的真正瓶颈。常见的质量属性分类如下：

表 2.4: 主要非功能需求类别与可度量示例

类别	关注点	示例度量
性能 (Performance)	响应时间、吞吐量、并发、资源占用	95% 请求 < 500 ms；峰值 2000 TPS
安全 (Security)	认证、授权、加密、审计、抗攻击	全链路 TLS；密码加盐哈希；等保三级
可用性/易用性 (Usability)	易学、易操作、可访问、可恢复	新用户 10 分钟内完成首单；WCAG 2.1 AA
可靠性 (Reliability)	平均无故障、故障恢复、容错	MTBF > 720 h；RTO < 30 min
可维护性 (Maintainability)	可修改、可测试、可复用、低耦合	圈复杂度 < 15；单元测试覆盖率 > 80%

- **可度量是硬要求：**非功能需求必须能写成带阈值、带条件的指标。「系统要快」不是需求；「在 4 核 8GB 环境下，95% 的商品查询响应时间 < 500 ms」才是需求。
- **术语辨析：**可用性 (Usability, 易用) 与可用性 (Availability, 可用/在线) 中文常同名，英文不同词，须结合上下文区分，避免混淆。
- **约束 (Constraint)** 是另一类特殊需求：如必须使用某数据库、必须符合某法规、预算与工期上限等。

2.4 软件需求规格说明

2.4.1 规格说明的特征

软件需求规格说明 (Software Requirements Specification, SRS) 是需求工程的正式产物（参见 ISO/IEC/IEEE 29148）。一份合格的 SRS 应具备七项特征，它们也可当作评审检查表：

表 2.5: SRS 应具备的七项特征

特征	含义	反例 / 检验
正确 (Correct)	真实反映干系人的需要	用户确认「这就是我要的」
无歧义 (Unambiguous)	每条只有一种解释	避免「适当」「快速」「友好」等模糊词
完整 (Complete)	涵盖全部功能、约束与边界	检查是否有「待定 (TBD)」遗漏
一致 (Consistent)	条目之间无冲突矛盾	两条需求不得给出相反的约束
可验证 (Verifiable)	能用测试或度量客观判定	「界面美观」不可验证；「响应 < 2s」可验证
可修改 (Modifiable)	结构清晰、条目独立、改一处不牵连全局	用编号与引用，避免重复描述
可追溯 (Traceable)	需求可双向链接到来源、设计、代码、测试	每条需求有唯一标识并可追溯到上游

2.4.2 SRS 的文档结构

SRS 的组织应服务于「可读、可查、可追溯」。一个经典的骨架如下：

表 2.6: SRS 的典型文档结构

章节	主要内容
1 引言	目的、范围、术语定义、参考文档、读者对象
2 总体描述	产品前景、用户特征、运行环境、约束、假设与依赖
3 具体需求	功能需求、外部接口、非功能需求、数据需求（逐条编号）
4 验证与追溯	需求的验收标准、追溯关系、与测试用例的映射
附录	原型图、词汇表、数据字典、原型或界面草图

- **良构 (well-formed)**: 每条需求应写明触发条件、主体、行为、对象、约束与验收标准，并原子化（一条只描述一件事），便于测试与追溯。
- **需求编号**: 为每条需求分配唯一标识（如 FR-07、NFR-03），这是追溯矩阵的基础。
- **反向需求**: 显式写出系统「不做什么」，避免范围被无限外扩。

2.5 用例建模

用例建模从**外部视角**描述系统「为谁提供什么服务」，是需求分析与用户沟通的核心工具。其基本元素包括参与者、用例以及它们之间的关系。

2.5.1 参与者、用例与关系

表 2.7: 用例图的构成元素

要素	图形	说明
参与者 (Actor)	火柴人	系统外部的用户或外部系统
用例 (Use Case)	椭圆	系统提供的一项完整、可交付价值的功能
系统边界 (Boundary)	矩形框	框内是用例，框外是参与者
关联 (Association)	实线	参与者与用例之间的通信
包含 (include)	虚线箭头 + <i>include</i>	基础用例 必然 执行被包含用例（公共步骤）
扩展 (extend)	虚线箭头 + <i>extend</i>	扩展用例在 特定条件 下插入基础用例
泛化 (Generalization)	空心三角	参与者或用例之间的继承（is-a）关系

表 2.8: include、extend 与泛化的方向与语义

关系	箭头方向	触发条件	语义
in-include	基础用例 → 被包含用例	必然执行	抽取多个用例共有的步骤（如「登录」「校验权限」）
ex-extend	扩展用例 → 基础用例	条件触发	给基础用例附加可选行为（如「申请退款」扩展「提交订单」）
泛化	子 → 父	—	子继承父的属性与行为；参与者泛化表示角色包含关系

2.5.2 用例图

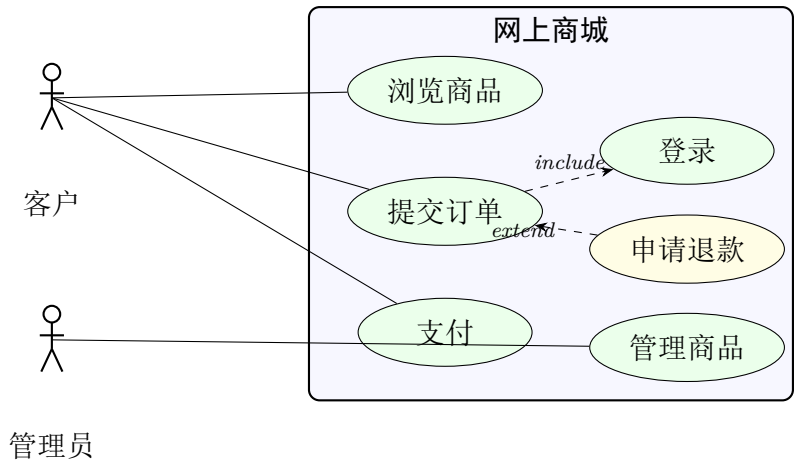


图 2.2: 用例图示例：网上商城的参与者、用例及 include/extend 关系（黄色用例为扩展用例）

include 与 extend 的区别：include 抽取的是**必然执行**的公共步骤，箭头由基础用例指向被包含用例；extend 附加的是**条件触发**的可选行为，箭头由扩展用例指向基础用例。二者方向相反，语义也相反。

2.5.3 用例描述模板

用例图只给出「有哪些用例」，每个用例的细节须用**用例描述 (Use Case Description)** 展开。标准模板包含前置条件、主流程、备选流程与后置条件：

表 2.9: 用例描述模板	
条目	内容
用例名称	动词短语，如「提交订单」
用例编号	唯一标识，便于追溯
参与者	触发该用例的主参与者与次参与者
概述	一两句话说明用例目标与价值
前置条件	用例开始前系统必须满足的状态
触发事件	启动该用例的事件或条件
主流程	正常路径的编号步骤序列
备选流程	异常或分支路径的处理
后置条件	用例结束后的系统状态与产出

2.5.4 用例描述示例

表 2.10: 用例描述示例：提交订单（UC-03）

条目	内容
用例名称	提交订单
用例编号	UC-03
参与者	主：客户；次：支付网关
概述	客户将购物车中的商品结算并生成订单，随后进入支付。
前置条件	客户已登录；购物车至少包含一件商品。
触发事件	客户在结算页点击「确认下单」。
主流程	1. 客户进入结算页，确认商品、数量与收货地址； 2. 系统计算商品金额、运费与可用优惠； 3. 客户选择支付方式并点击「确认下单」； 4. 系统校验库存与地址合法性； 5. 系统生成订单，状态置为「待支付」并锁定库存； 6. 系统跳转支付网关，客户完成支付； 7. 支付成功后，系统将订单置为「已支付」并发送通知。
备选流程	A1 库存不足：第 4 步若库存不足，提示缺货并允许修改数量或删除商品； A2 地址非法：校验失败时返回第 1 步要求重新填写； A3 支付超时：15 分钟内未支付，订单自动取消并释放库存。
后置条件	成功：生成一条「已支付」订单，库存扣减，产生支付流水； 失败：订单为「待支付」或「已取消」，库存不扣减或已回滚。

2.6 需求管理

需求一旦开始，就会不断变化。需求管理的目的不是「禁止变化」，而是让变化可控、可见、可追溯。

2.6.1 基线与变更控制

基线 (Baseline) 是需求通过评审后冻结下来的一个版本，作为后续开发、测试与验收的共同基准。基线并非不可变，而是「变更必须走流程」。

变更控制 (Change Control) 用来防止需求无序蔓延 (scope creep)。其典型流程为：

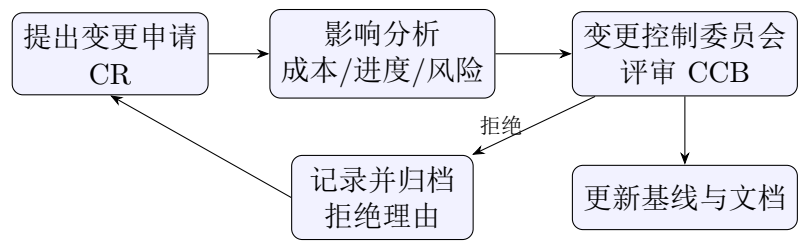


图 2.3: 需求变更控制流程：申请 → 影响分析 → CCB 评审 → 批准并更新基线，或拒绝并归档

- **变更申请 (Change Request, CR)**：记录变更内容、提出人与理由。
- **影响分析 (Impact Analysis)**：评估对成本、进度、质量、其他需求与已有实现的影响，依据追溯矩阵定位波及范围。
- **变更控制委员会 (Change Control Board, CCB)**：由业务、开发、测试等代表组成，负责批准或拒绝。
- 一个常见误区：口头答应变更、不走流程，导致「文档说一套、代码做一套」，追溯彻底失效。

2.6.2 需求追溯矩阵

需求追溯矩阵 (Requirements Traceability Matrix, RTM) 在需求、设计、代码与测试用例之间建立双向链接，用于保证覆盖完整、支持影响分析与验收。其两个方向为：

- **前向追溯**：由需求 → 设计 → 编码 → 测试，检查每条需求是否都被实现与验证。
- **后向追溯**：由测试/代码 → 需求，检查是否存在「无需求来源」的镀金功能。

	设计	编码	测试
FR1	✓	✓	✓
FR2	✓	✓	✓
FR3	✓		✓
FR4	✓		

图 2.4: 需求追溯矩阵示例：✓ 表示该需求已由对应设计、编码或测试覆盖；FR3 缺少编码覆盖、FR4 缺少编码与测试覆盖，暴露追溯缺口

表 2.11: 追溯矩阵的常见用途

用途	说明
覆盖检查	找出未被任何测试覆盖的需求，防止漏测
镀金清查	找出没有需求来源的代码或功能，防止过度实现
影响分析	变更某需求时，快速定位受影响的模块与用例
验收依据	逐条对照需求与测试，支撑验收与审计

2.7 本章小结

- 需求工程五项活动是**获取** → **分析** → **规格说明** → **验证** → **管理**，且高度迭代，管理贯穿始终。
- 获取方法（访谈、问卷、观察、原型、文档分析、JAD）各有取舍，宜组合并交叉验证。
- 需求分为**功能需求**与**非功能需求**；后者必须可度量，性能、安全、易用、可靠、可维护是常见质量属性。
- SRS 应满足**正确、无歧义、完整、一致、可验证、可修改、可追溯**七项特征，并逐条编号。
- 用例建模从外部视角刻画系统服务，注意 include（必然）、extend（条件）与泛化的区别；用例描述模板补齐前置/主流程/备选/后置。
- 需求管理通过**基线、变更控制、追溯矩阵**让变化可控、可查、可追溯。

2.8 常见误区

- **把设计混入需求**：需求回答「做什么」，若写成「用 Redis 缓存实现」就把解空间当成了问题域，限制了设计自由。
- **需求镀金 (Gold Plating)**：实现用户并未要求的功能，看似「贴心」，实则增加成本、风险与维护负担，且常常无需求来源。
- **需求蔓延 (Scope Creep)**：不做影响分析就不断「顺手加一点」，导致进度与预算失控。
- **不可度量的非功能需求**：如「系统要快、要安全、要友好」，无法验证，等价于没有需求。
- **遗漏干系人与隐含需求**：只看直接用户，忽略运维、法务、接口方，导致上线后大量返工。
- **用例粒度过细**：把用例写成逐步 UI 操作或设计细节，既难维护，也越界到了设计。
- **需求与测试脱节**：缺少追溯矩阵，验收时说不清「这条需求到底测没测」。

第三章 结构化分析与设计

结构化方法从「数据流」与「模块」两个视角刻画系统：用数据流图（DFD）描述数据如何流动与变换，用数据字典精确定义数据，再用「高内聚、低耦合」的模块化原则指导设计。本章建立「DFD → 数据字典 → 模块化」的分析与设计主线。

3.1 结构化分析思想

3.1.1 为什么需要结构化

软件需求往往以自然语言叙述，而自然语言是歧义、冗余、含糊的：同一个词在不同段落可能指不同事物，一个「系统」既可能指整个业务，也可能指其中一段处理。若直接据此编码，需求中的矛盾要到集成测试甚至上线才暴露，返工代价极高。结构化分析（Structured Analysis, SA）的出发点，就是用一组受控的抽象（数据流与数据定义）把「做什么」从「怎么做」中剥离出来，使需求在动手实现之前就能被检查、被确认、被追踪。

与面向对象方法把系统看作「相互协作的对象」不同，结构化方法把系统看作对数据的一系列变换：数据从外部进入系统，被逐步加工，再送回外部或被持久化。这种「以数据为中心」的视角，特别适合事务处理类、信息管理类系统。

3.1.2 三条核心原则

- **自顶向下（top-down）**：从系统的整体功能出发，先看系统与外界交互的轮廓，再深入内部细节。避免一开始就陷入局部实现而失去全局。
- **逐层分解（stepwise refinement）**：把复杂问题逐层拆成规模更小、更容易理解与处理的子问题，直到每个子问题都足够简单、可直接实现为止。
- **数据与处理分离**：分别刻画「数据是什么（数据字典）」与「数据被如何处理（DFD 中的加工）」，两者在图上并列但概念上独立。这样数据定义的变更与处理逻辑的变更可分别管理。

表 3.1: 结构化方法在生命周期各阶段的产物

阶段	主要活动	核心产物
需求分析	建立功能模型，划分数据与处理	数据流图（分层）、数据字典
总体设计	从 DFD 导出模块结构	结构图（模块层次 + 调用）
详细设计	细化模块内部算法与数据	模块说明、伪代码、数据库设计

例：一个「图书借阅系统」，需求原文可能是「读者可以借书，管理员可以入库新书」。结构化分析首先要回答：借书时数据从哪里来、经哪些加工、写到哪里去；「图书」这个数据项究竟包含哪些字段。这些正是 DFD 与数据字典要固化的内容。

3.2 数据流图（DFD）

数据流图（Data Flow Diagram, DFD）从数据流动的角度描述系统「做什么」：只关心数据及其变换，不关心控制时序与实现细节。它是一种**面向数据**的功能模型，图中没有分支、循环、先后次序等控制语义。

3.2.1 四种基本元素

表 3.2: DFD 的四种基本元素

元素	图形	说明
外部实体	矩形	系统边界外的参与者（人 / 组织 / 外部系统），是数据的源或宿
加工	圆 / 椭圆	对数据执行变换的处理单元，命名用「动词 + 名词」
数据存储	开口矩形	数据的静态仓库（文件 / 表 / 库），只能由加工读写
数据流	带箭头的线	沿箭头方向流动的数据，命名用名词，不代表控制或时序

使用四种元素时有几条约定：**外部实体之间不直接画数据流**（它们必须经加工交互）；**数据存储之间也不直接画数据流**（读写只能由加工发起）；每条数据流的名字应是名词短语（如「订单」而不是「提交订单」）。命名若含混，往往意味着对需求的理解还不清楚。

3.2.2 0 层与 1 层分解

0 层 DFD（上下文图, context diagram）：把整个系统看作一个加工，只画该系统与所有外部实体之间的输入 / 输出数据流，用来界定**系统边界**与外部接口。上下文图不出现数据存储。

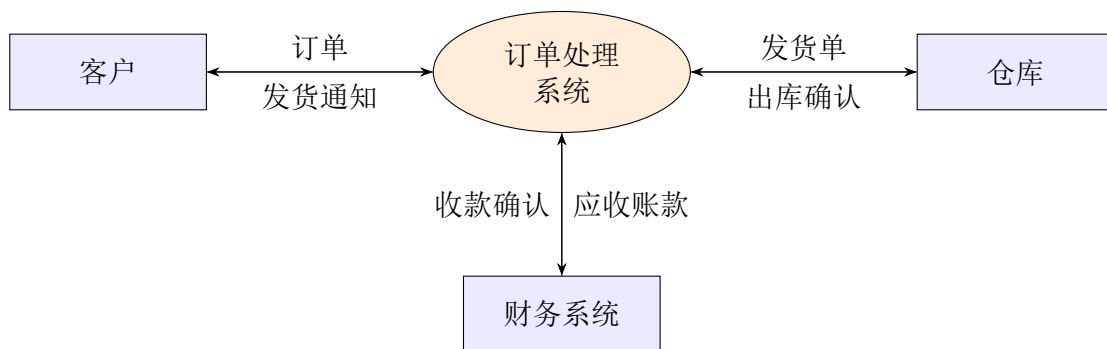


图 3.1: 0 层 DFD：订单处理系统的上下文图（只画系统与外部实体的数据交换）

1 层 DFD：把 0 层那个唯一的加工**分解**为若干子加工，并画出与之相关的数据存储。下面是订单处理系统的 1 层分解。

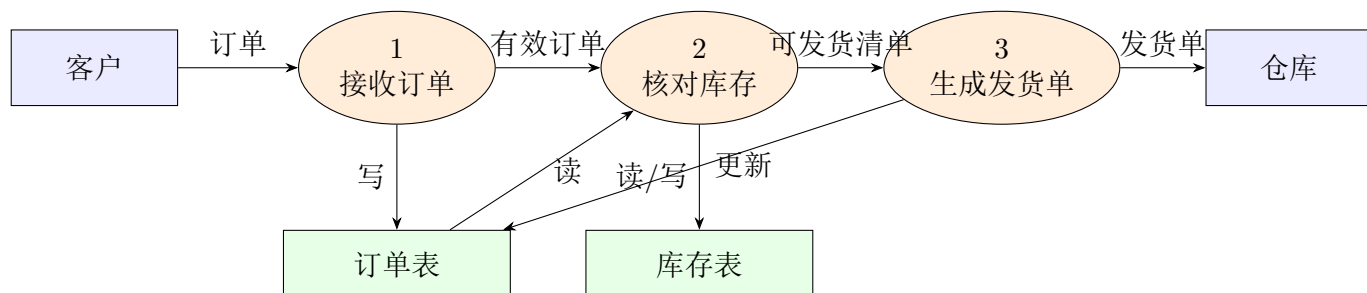


图 3.2: 1 层 DFD：订单处理系统分解（加工编号 1、2、3，并引入数据存储）

对某个子加工再分解，就得到 2 层、3 层 ... DFD。层次不宜过深，一般到 3 层左右即可，再细就应交给详细设计。

3.2.3 父子图平衡原则

平衡 (balance) 是 DFD 分层最关键的约束：

- **数据守恒：**子图边界上所有输入数据流，其集合必须与父图中对应加工的输入数据流一致；输出同理。分解时数据既不凭空产生，也不凭空消失。
- **一一对应：**父图加工的一条输入流，在子图中可拆成多条（细化），但这些细流的**来源**仍须是被分解加工的外部来源；关键在于「进入 / 离开该加工的数据总量与名称在概念上守恒」。
- **编号一致：**加工编号体现层次，父加工 1 分解出的子加工编号为 1.1、1.2 ...；引用的数据存储与数据流名称须在数据字典中有定义。

例如上下文图中「订单处理系统」的输出有「发货单」和「发货通知」，则 1 层子图的输出中必须能找到它们对应的数据流，否则失衡。

3.2.4 一个完整的小例子

以「ATM 取款」为例：外部实体是「储户」和「银行主机」；系统只有 0 层一个加工。1 层分解为「1 验卡、2 验密、3 处理取款、4 记账」。其中「3 处理取款」的输入是「取款金额」，输出是「现金」；父图中 ATM 输出给储户的「现金」必须由子图某加工产生，这就是平衡的应用。

3.3 数据字典

数据字典 (Data Dictionary, DD) 为 DFD 中的每个数据流、数据存储与数据项给出**精确定义**，是 DFD 的「词条说明」。DFD 与 DD 互补：图给出**结构与流向**，字典给出**内容与组成**。

3.3.1 记号的语义

表 3.3: 数据字典常用记号		
记号	含义	例
=	被定义项 = 其组成	订单 = 订单号 + 客户 + {订单项}
+	顺序连接 (逻辑与)	客户 = 姓名 + 地址 + 电话
{ }	重复 (0 次或多次)	{订单项}
[]	从中选择一项	性别 = [男 女]
	在 [] 内分隔备选项	男 女
()	可选 (可有可无)	电话 = 座机 + (手机)

记忆要点： + 表示「并且都要」， [] 表示「从里面挑一个」， { } 表示「重复若干个」， () 表示「可要可不要」。把记号当作一种**正规文法**的缩写来读，就能写出精确的数据结构。

3.3.2 一个完整条目示例

一份订单的数据字典可以写成：

订单 = 订单号 + 客户 + {订单项} + (备注)
订单号 = 10 位数字
客户 = 客户号 + 姓名 + 收货地址 + (联系电话)
订单项 = 商品号 + 数量 + 单价
收货地址 = 省 + 市 + 街道 + 门牌号
数量 = 1 到 999 的整数

读第一条即：一张订单由一个订单号、一个客户、零到多个订单项、以及可选的备注组成。数据存储的条目也可同样定义，例如：

订单表 = {订单}
库存表 = {商品号 + 当前数量 + 安全库存}

一致性要求：DFD 中出现的每一个名字（数据流、数据存储），都必须在字典中有且仅有一条定义；反之字典中定义的名词，也应在某张 DFD 中出现。借助字典还能做**数据平衡检查**：用记号推导出的数据结构，可作为数据库表设计与测试用例设计的依据。

3.4 结构化设计：模块、耦合与内聚

结构化设计（Structured Design, SD）把分析阶段的 DFD 转换成设计阶段的**模块结构图**（structure chart）。模块化把系统划分为相对独立、单一职责的模块，其质量由**耦合**（模块之间联系的紧密程度）与**内聚**（模块内部各成分结合的紧密程度）共同衡量，目标是**高内聚、低耦合**。

3.4.1 模块结构图的量与术语

表 3.4: 模块结构图的常用度量

术语	定义	设计倾向
模块	可独立命名、调用、测试的功能单元	职责单一
扇出 (fan-out)	一个模块直接调用的下层模块个数	一般 3 ~ 7，过大难维护
扇入 (fan-in)	直接调用某模块的上层模块个数	越大表示复用越好
深度 (depth)	模块层次的层数	反映控制的层次，过深难理解
宽度 (width)	同一层中模块的最大个数	反映控制的跨度，过宽难协调

扇出过大意味着一个模块管得太多，扇入大说明该模块被多个上层复用，是我们追求的方向。

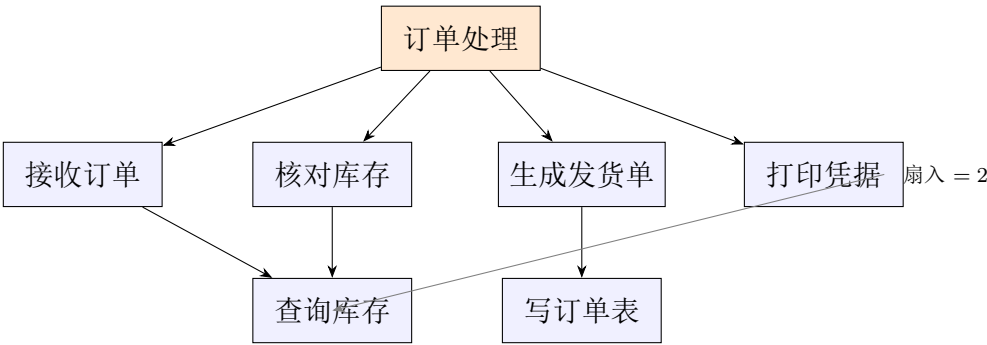


图 3.3: 模块结构图示例：订单处理（根模块扇出为 4，查询库存模块的扇入为 2）

3.4.2 耦合（Coupling）

耦合由低到高依次为：非直接耦合、数据耦合、标记耦合、控制耦合、外部耦合、公共耦合、内容耦合。**耦合越低越好**。

表 3.5: 耦合类型（从低到高，越靠上越好）

类型	传递内容	判定	说明
非直接耦合	无	两模块无直接联系	最好，模块完全独立
数据耦合	简单数据	参数只是基本数据项	最常用的良好耦合
标记耦合	数据结构	传整个记录，只用到其中一部分	造成不必要的数据依赖
控制耦合	控制信息	传开关 / 标志控制对方逻辑	增加理解与修改难度
外部耦合	外部约束	共同受外部格式 / 协议约束	如都依赖某文件格式
公共耦合	公共数据区	共享全局变量	牵一发而动全身
内容耦合	直接访问内部	一个模块直接访问另一模块内部数据 / 代码	最差，应彻底避免

示例：模块 A 调用 B 时传「一个整数」——数据耦合；传「一整条记录而 B 只用其中 ‘name’ 字段」——标记耦合；传「布尔开关 ‘isPrint’ 让 B 决定是否打印」——控制耦合；A 与 B 通过全局变量 ‘gCount’ 通信——公共耦合；A 直接 ‘goto’ 到 B 内部标签或读写 B 的私有变量——内容耦合。

3.4.3 内聚（Cohesion）

内聚由高到低依次为：功能内聚、顺序内聚、通信内聚、过程内聚、时间内聚、逻辑内聚、偶然内聚。内聚越高越好。

表 3.6: 内聚类型（从高到低，越靠上越好）

类型	结合方式	判定	说明
功能内聚	单一功能	所有成分共同完成一个明确功能	最好，理想模块
顺序内聚	数据顺序	上一步输出即下一步输入	数据顺序流动，联系紧密
通信内聚	同一数据	各成分操作同一输入 / 输出数据	操作同数据但无顺序
过程内聚	执行顺序	按固定流程顺序执行，数据联系弱	仅靠流程绑定
时间内聚	同一时段	因需同时段执行而聚合	如初始化全部变量
逻辑内聚	逻辑相似	逻辑同类，由参数选择执行分支	多功能揉在一起
偶然内聚	无关系	各成分之间无明显关系	最差，无从命名

示例：一个模块「计算订单总价」只做一件事——功能内聚；「读文件 → 解析每行」——顺序内聚；「初始化界面、初始化数据库连接」同属启动——时间内聚；把「打印、复制、排序」都塞进一个‘doEverything(type)’ 由参数选择——逻辑内聚；毫无关系、只为省事的几行代码凑一起——偶然内聚。

3.4.4 高内聚、低耦合

理想的模块划分是「功能内聚 + 数据耦合」：模块内部只做一件事且各成分紧密协作，模块之间只通过简单数据进行必要的通信。这样的模块**易理解、易测试、易复用、易维护**——修改一个功能不必担心波及无关模块，替换一个实现也不必改动调用方。判断一个划分是否合理，可以问三个问题：这个模块能用一句话说清职责吗（内聚）？它需要的输入能否只用最简单的数据表达（耦合）？把它拿掉后其它模块还能独立工作吗（独立性）？

3.5 变换分析与事务分析

从 DFD 到结构图，并没有唯一答案，但有两种经典的映射策略。它们都先把 DFD 重新审视，找出输入流、变换中心、输出流的结构。

3.5.1 变换分析（Transform Analysis）

当 DFD 呈「输入 → 变换 → 输出」的线性形态时，可用变换分析。步骤如下：

- 找出 DFD 中的**变换中心**（真正改变数据形态的那部分加工）；
- 沿变换中心划出**输入流**（把外部数据变换为中心可用的形式）与**输出流**（把结果变换为外部需要的形态）；
- 为变换中心建一个上层模块，为输入、输出各建一个分支，再逐层细化。

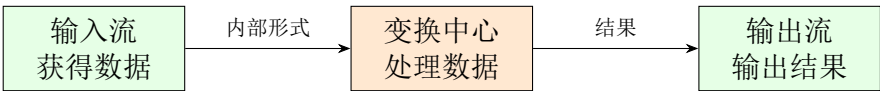


图 3.4: 变换分析的 DFD 形态：输入流 → 变换中心 → 输出流

3.5.2 事务分析 (Transaction Analysis)

当 DFD 中某加工根据**事务类型**决定走不同处理路径时，用事务分析。它先把事务中心识别出来，再为每一种事务建立一条处理分支，每条分支撑从「输入」到「完成」的完整路径。

示例：一个「单据处理」加工，按单据类型（入库单、出库单、退货单）分别调用不同处理流程，就是典型的事务结构。事务分析适合**多分支、分支结构相似**的场景。

变换 vs. 事务：变换分析处理的是「一条流水线」，事务分析处理的是「一个分发器下挂多条流水线」。实际系统中两者常混合使用：外层是事务分析（按请求类型分发），每条分支内部用变换分析（走输入—处理—输出）。

3.6 本章小结与常见误区

3.6.1 本章小结

- 结构化分析的三条原则：自顶向下、逐层分解、数据与处理分离。
- DFD 用外部实体、加工、数据存储、数据流四种元素描述数据处理过程；0 层是上下文图，1 层起逐层分解，必须满足父子图平衡。
- 数据字典用 =, +, { }, [], |, () 等记号精确描述数据组成，与 DFD 交叉印证。
- 结构化设计从 DFD 导出模块结构图，用扇入 / 扇出、深度 / 宽度度量结构，用耦合（越低越好）与内聚（越高越好）评价模块质量。
- 映射策略有变换分析与事务分析两种，常混合使用。

3.6.2 常见误区

表 3.7: 结构化分析与设计中的常见误区

误区	纠正
DFD 里画流程箭头表示先后 / 判断分支	DFD 只表达数据变换，控制时序属于流程图或状态图
外部实体之间直接连数据流	外部实体必须经加工交互，不能直连
分解后子图数据流凭空出现或消失	必须遵守父子图平衡，数据总量守恒
数据字典与 DFD 各写各的	两者名字必须一致，可交叉检查
只看内聚或只看耦合	两者需同时权衡，目标是高内聚 且 低耦合
扇出越大、模块越细越好	扇出过大难维护，独立性与职责单一才是目标

第四章 面向对象与 UML

面向对象方法以「对象」为核心组织软件，UML 用一组标准图从**结构**（类图、用例图）与**行为**（时序图、状态图、活动图）两个视角刻画系统。本章建立「类-关系-交互-状态」的建模主线。

4.1 类图

类图是 UML 中最核心的**结构图**，描述系统的静态结构：类、接口以及它们之间的关系。一个类通常画成**三格矩形**：类名、属性、方法。

4.1.1 类与成员

表 4.1: 成员的可见性修饰符

符号	可见性	含义	示例
+	public（公有）	任何类都可访问	+ getName(): String
-	private（私有）	仅本类可访问	- balance: Money
#	protected（受保护）	本类及子类可访问	# id: String
~	package（包）	同包内可访问	~cache: Map

成员书写的通用格式为「**可见性名称: 类型 [多重性] = 默认值**」；方法为「**可见性名称 (参数): 返回类型**」。静态成员加**下划线**，抽象成员用**斜体**（或类名用 {abstract} 标注）。

4.1.2 类之间的关系

表 4.2: 六种类关系的符号与语义

关系	符号	语义	耦合
关联 (Association)	实线（可带方向箭头）	类之间的一般连接，如「教师—课程」	弱
聚合 (Aggregation)	空心菱形 + 实线（菱形在整体端）	整体—部分，部分可独立存在（has-a）	中
组合 (Composition)	实心菱形 + 实线（菱形在整体端）	整体—部分，部分随整体消亡（owns-a）	强
泛化 (Generalization)	空心三角 + 实线（三角指向父类）	继承关系 is-a，子类复用父类	强
依赖 (Dependency)	虚线 + 开放箭头	临时使用，如参数、局部变量、返回值	弱
实现 (Realization)	虚线 + 空心三角（指向接口）	类实现接口所定义的契约	中

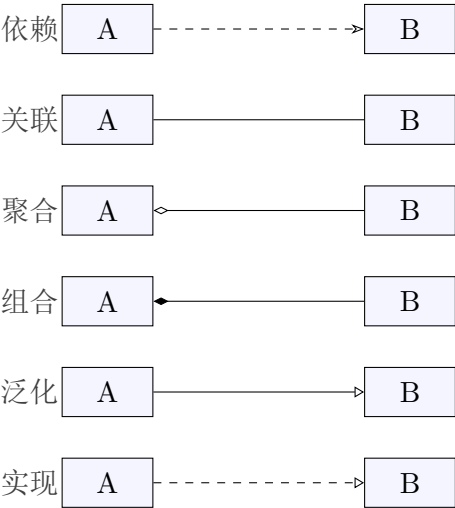


图 4.1: 六种类关系的图形符号（关联/聚合/组合以 A 为整体端；泛化/实现以 B 为父类或接口端）

4.1.3 多重性

多重性 (multiplicity) 标注在关联线的两端，表示一个对象在给定时刻可与多少个对端对象相关联。

表 4.3: 常用的多重性标注

标注	读法	示例
1	恰好一个	一张订单只属于一个客户
0..1	零或一个	可选引用
*	零或多个	一个客户可有任意多张订单
1..*	至少一个	订单至少包含一个订单项
n..m	区间	例如 2..4 个玩家

4.1.4 类图示例

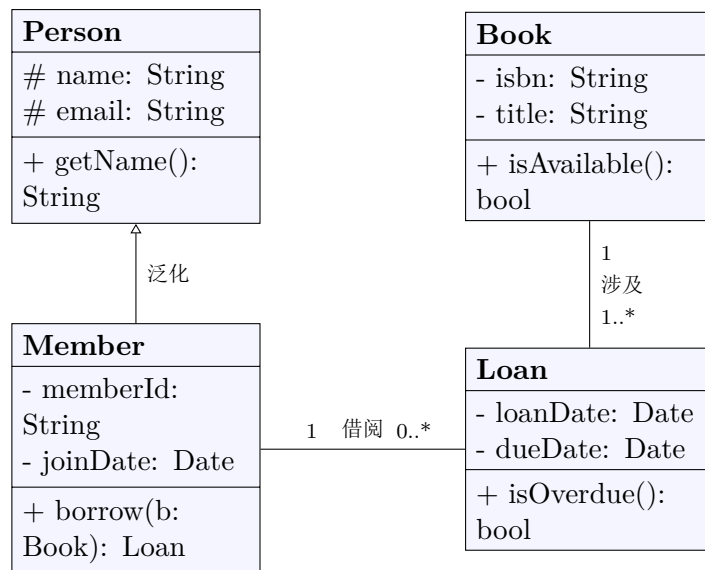


图 4.2: 类图示例：图书馆借阅系统（Member 继承 Person，借阅记录 Loan 关联 Member 与 Book）

- 泛化：Member 是 Person 的子类，继承 name、email 等成员。
- 关联 + 多重性：一个 Member 可产生 0..* 条 Loan；每条 Loan 恰好对应 1 个 Member。
- 属性/方法的类型：String、Date、bool 为数据类型，不参与对象间关联。

4.2 用例图

用例图从外部视角描述系统「为谁提供什么服务」，是需求分析阶段与用户沟通的主要工具。

4.2.1 要素与关系

表 4.4: 用例图的主要要素

要素	图形	说明
参与者 (Actor)	火柴人	系统外部的用户或外部系统
用例 (Use Case)	椭圆	系统提供的一项完整功能
系统边界 (Boundary)	矩形框	框内是用例，框外是参与者
关联 (Association)	实线	参与者与用例的通信
包含 (include)	虚线箭头 + <i>include</i>	被包含用例是必选的公共步骤
扩展 (extend)	虚线箭头 + <i>extend</i>	扩展用例在特定条件下才插入
泛化 (Generalization)	空心三角	参与者或用例的继承

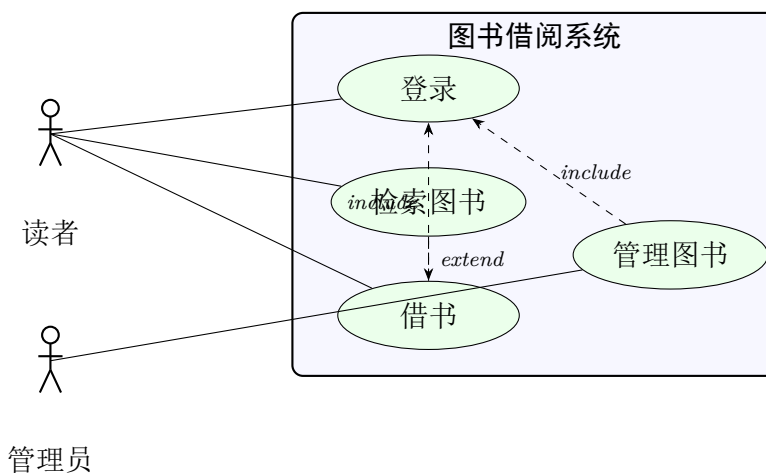


图 4.3: 用例图示例：读者与管理员使用图书借阅系统

include 与 extend 的区别：include 是把多个用例共有的步骤抽成公用例（**必然执行**）；extend 是给基础用例附加可选行为（**条件触发**）。箭头方向也不同：include 由基础用例指向被包含用例，extend 由扩展用例指向基础用例。

4.3 时序图

时序图（顺序图）是交互图，强调消息在时间上的先后顺序，用于描述一个用例内部对象之间的协作。

4.3.1 要素

表 4.5: 时序图的主要要素

要素	图形	说明
生命线 (Lifeline)	参与者下方的虚线	对象在时间轴上的存在
激活条 (Activation)	生命线上的窄矩形	对象正在执行（控制焦点）
同步消息 (Synchronous)	实线 + 实心箭头	调用后等待返回，阻塞
异步消息 (Asynchronous)	实线 + 开放箭头	发送后不等待，继续执行
返回消息 (Return)	虚线 + 开放箭头	把结果返回给调用者
自消息 (Self)	折返箭头	对象调用自身方法

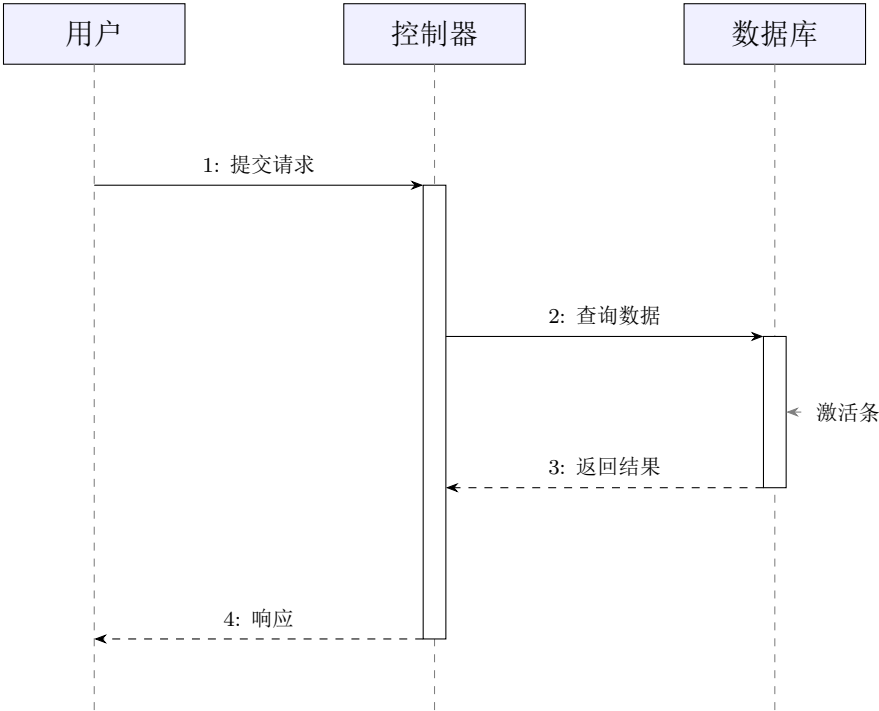


图 4.4: 时序图示例：一次查询交互（实线为同步消息，虚线为返回消息）

同步与异步：同步消息的发送者在收到返回前被阻塞，通常配套一个返回消息；异步消息发送后立即继续，不等待返回，常用于事件通知。激活条的高度反映对象执行该操作的时长。

4.4 状态图与活动图

4.4.1 状态图

状态图描述单个对象在其生命周期内状态的变化，转移的通用语法为：

事件 [守卫] / 动作

(4.1)

即「发生某事件，在守卫条件成立时，执行某动作并迁移到目标状态」。

表 4.6: 状态图的主要要素

要素	图形	说明
状态 (State)	圆角矩形	对象所处的条件或状况
初始状态	实心圆	唯一的入口
终止状态	牛眼（圆内实心圆）	生命周期结束
转移 (Transition)	带箭头实线	状态之间的迁移
事件 (Event)	转移上的触发名	触发转移的信号
守卫 (Guard)	[条件]	转移能否发生的布尔条件
动作 (Action)	/动作	转移发生时执行的行为

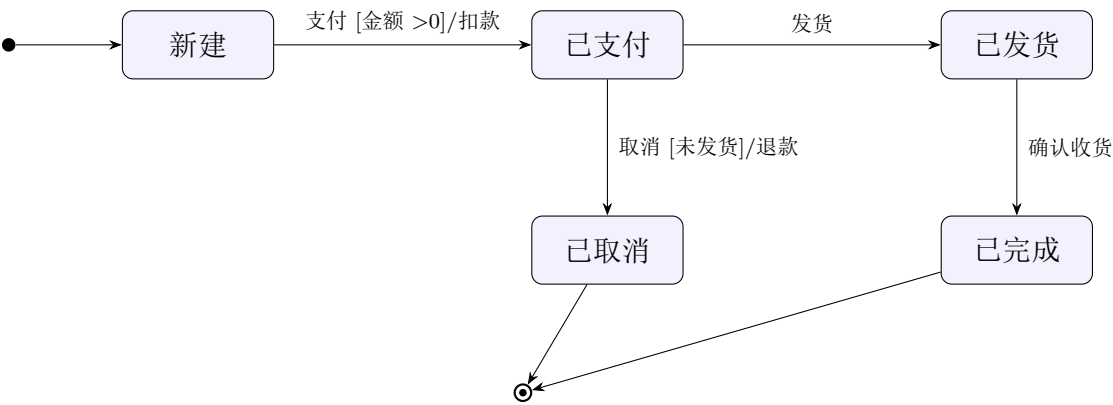


图 4.5: 订单状态图：事件 [守卫]/动作

4.4.2 活动图

活动图描述工作流或算法流程，与流程图相近，但支持并发、泳道等语义。

表 4.7: 活动图的主要要素

要素	图形	说明
开始 (Initial)	实心圆	流程入口
活动 (Action)	圆角矩形	一个可执行的步骤
判断 (Decision)	菱形	依据条件选择分支
合并 (Merge)	菱形	多条分支汇合
分叉/汇合 (Fork/Join)	粗黑线	并发流的开始与同步
泳道 (Swimlane)	分区	按负责对象划分活动
结束 (Final)	牛眼	流程终止

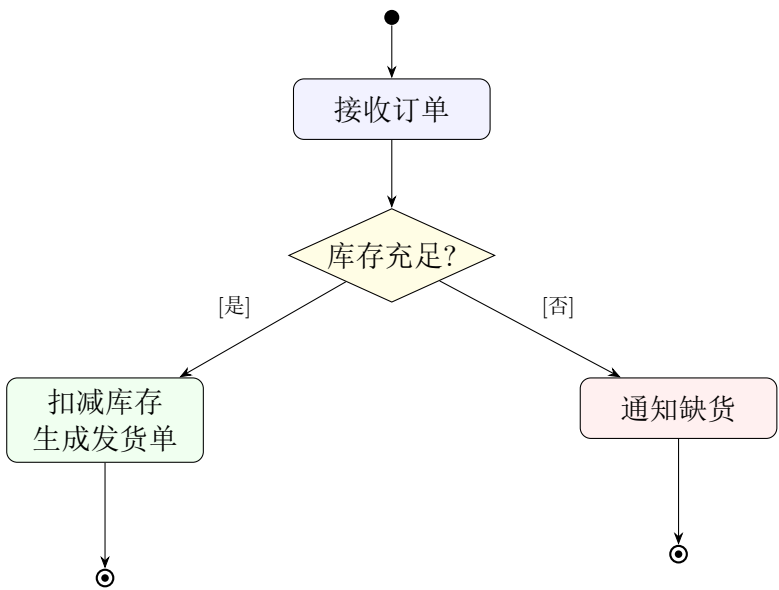


图 4.6: 活动图示例：订单处理流程（含判断分支）

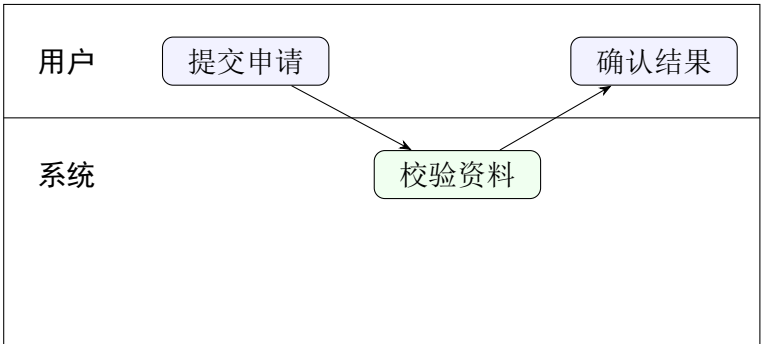


图 4.7: 泳道活动图：按「用户/系统」分区，箭头跨泳道表示职责交接

状态图 vs. 活动图：状态图刻画一个对象因事件驱动的状态变迁（关注「对象处于什么状态」）；活动图刻画一个流程中动作与控制的流转（关注「下一步做什么」）。二者都可建模行为，但抽象对象与关注点不同。

第五章 数据建模与 ER

数据建模把现实世界抽象为「实体-联系-属性」，再用 ER 图与关系模式两种语言描述同一份语义。本章从 ER 的基本构件出发，给出到关系模式的转换规则，并用函数依赖与范式刻画「好」的模式。

5.1 数据建模概述

数据建模是数据库概念设计阶段的核心活动。现实世界的对象先经过人的认识抽象为**信息世界**的概念模型，再转换为**机器世界**的逻辑与物理模型。ER 模型正是信息世界的描述工具，关系模型则是机器世界最常用的逻辑模型。二者描述的是同一份语义，只是抽象层次与表达形式不同。

表 5.1: 三个世界与相应的建模术语

层次	建模对象	术语示例
现实世界	客观事物及其相互联系	一名学生、一门课程、学生选课
信息世界	概念模型（ER 模型）	实体、属性、联系
机器世界	逻辑模型（关系模型）	表、行（元组）、列（属性）

数据建模通常分三步：**需求理解**（搞清有哪些对象、哪些规则）→ **概念建模**（画出 ER 图，独立于任何 DBMS）→ **逻辑建模**（用转换规则得到关系模式）。ER 图的价值在于它贴近人的思维、便于与用户沟通；关系模式的价值在于它能被数据库系统直接实现。本章的主线就是「用 ER 抓住语义，再用规则把语义无损地搬到关系表中」。

5.2 实体、属性与键

5.2.1 实体与实体集

实体（Entity）是客观存在且可相互区分的事物，既可以是具体对象（一名学生、一本图书），也可以是抽象概念（一门课程、一次选课）。具有相同性质的实体构成**实体集**（Entity Set），如「全体学生」。在 ER 图中，实体集用**矩形**表示。

5.2.2 属性的分类

属性（Attribute）是实体具有的特征。按结构与取值方式，属性分为四类，见表 5.2。

表 5.2: 属性的四种分类		
类型	含义	示例（对应 ER 记号）
简单属性	不可再分的最小单位	学号、性别（单椭圆）
复合属性	可分解为若干子属性	姓名（姓、名）、地址（省、市、街道）
多值属性	同一实体可取多个值	电话、学位（双椭圆）
派生属性	由其他属性计算得到	年龄（由出生日期派生，虚椭圆）

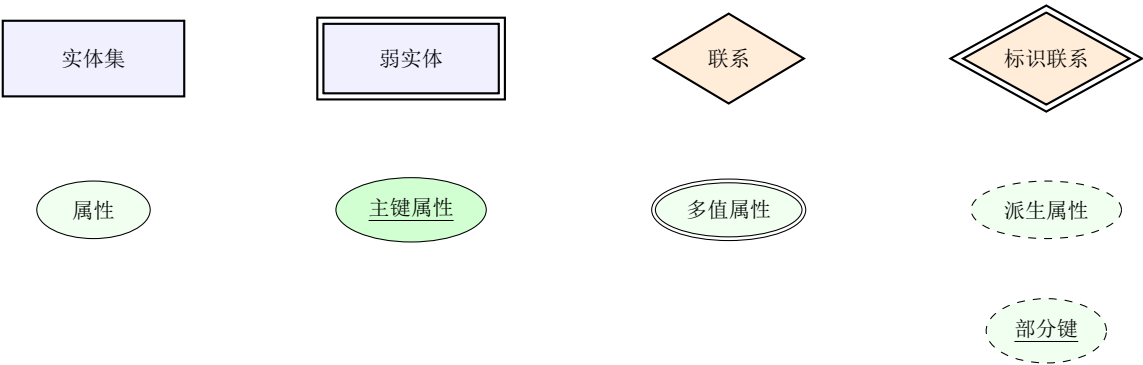


图 5.1: ER 图符号：矩形 = 实体集，双矩形 = 弱实体，菱形 = 联系，双菱形 = 标识联系，椭圆 = 属性，下划线 = 主键，双椭圆 = 多值属性，虚椭圆 = 派生属性或部分键；实体端双线表示全参与

复合属性允许我们保留自然的分组，但落到关系表时必须展平为若干简单列。多值属性会破坏第一范式，必须为其单独建表。派生属性一般**不存储**，以免与源属性不一致；只有在性能需要时才冗余存储，并用触发器或程序维护。

5.2.3 键

键（Key）是能唯一标识实体的属性或属性组，是实体集与其属性之间最重要的约束。各类键的含义见表 5.3。

表 5.3: 键的分类

键	含义	示例
超键 (Superkey)	能唯一标识元组的属性组（可含冗余属性）	{学号, 姓名}
候选键 (Candidate Key)	最小的超键，去掉任一属性都不能唯一标识	{学号}、{身份证号}
主键 (Primary Key)	从候选键中指定的一个，非空、唯一	学号
外键 (Foreign Key)	引用另一关系主键的属性（组），可为空	选课. 学号
部分键 (Partial Key)	弱实体内用于区分同类实体的属性	订单项的「行号」

超键一定包含**候选键**；**候选键**是极小的超键，一个实体集可以有多个候选键；**主键**是设计者从中选定的、用于日常标识的那一个（在 ER 图中加下划线）。**外键**不是一个表内部的唯一性约束，而是表与表之间的引用约束，其取值要么等于被引用主键的某个值，要么为空（空值是否允许取决于参与约束）。

5.2.4 弱实体

弱实体（Weak Entity）自身没有完整主键，必须依附于某个**强实体**（标识实体，Identifying Entity）而存在。弱实体用来区分同类实体的属性称为**部分键**，它只在本实体内唯一，脱离标识实体则无法唯一标识。例如「订单项」依附于「订单」，同一订单内的行号 1、2、3 可区分，但不同订单都可能行号 1。

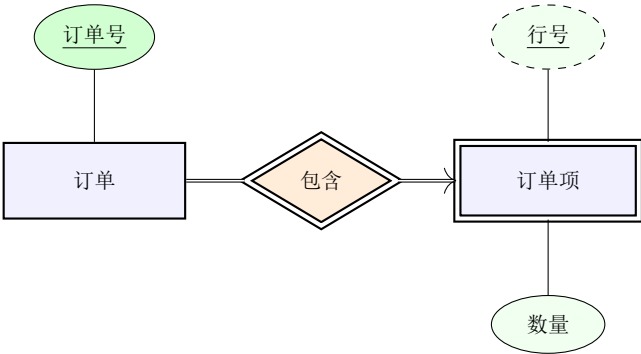


图 5.2: 弱实体示例：订单项依附于订单。强实体与标识联系用双线，弱实体用双矩形，部分键用虚线下划线椭圆

弱实体的主键由「标识实体的主键 + 自身的部分键」组成，即订单项的主键为 {订单号, 行号}。这一事实决定了它在转换成表时如何构造主键。

5.3 联系

5.3.1 联系的度

联系（Relationship）刻画实体之间的语义关联。按参与实体的个数，联系分为**一元**（递归）、**二元**、**三元**，见表 5.4。

表 5.4: 联系的度（度数）

度	含义	示例
一元（递归）	同一实体集内部两实体相关联	员工「管理」员工
二元	两个实体集相关联	学生「选修」课程
三元	三个实体集同时相关联	供应商「供应」项目-零件

三元联系不能被拆成三个二元联系，因为它的语义是「三者作为一个整体组合一次」。例如「红旗公司向桥梁项目供应螺丝」这条事实只有在三个参与者同时确定时才成立。

5.3.2 基数

基数（Cardinality）描述联系的映射比例，即一端的一个实体最多/恰好对应另一端的多少个实体：

- 1:1：甲至多对应一个乙，乙也至多对应一个甲。例如「班级」与「班长」。
- 1:N：甲对应多个乙，乙至多对应一个甲。例如「院系」与「学生」。
- M:N：甲对应多个乙，乙也对应多个甲。例如「学生」与「课程」。

在 ER 图中，基数常写在联系线与实体集相接处（1、N、M）。

5.3.3 参与约束

参与约束规定实体集中的实体是否必须参与某联系：

- **全部参与**（Total，存在依赖）：实体集中每个实体都必须参与该联系，图中用**双线**。
- **部分参与**（Partial）：允许某些实体不参与，图中用**单线**。

基数刻画「最多多少」，参与约束刻画「至少多少（是不是零）」，二者正交。例如「每个学生必须选修至少一门课」说明学生侧**全参与**；而一门课可以暂时无人选修说明课程侧部分参与。

5.3.4 属性化联系

属性化联系指联系自身带有属性。例如「选修」带有「成绩」，「供应」带有「供应量」。当联系为 $M:N$ 时，这些属性在转换时恰好随中间表保存；当联系为 $1:N$ 时，应把属性放到 N 端的关系中。属性化联系是 ER 建模区别于普通图模型的重要特征。

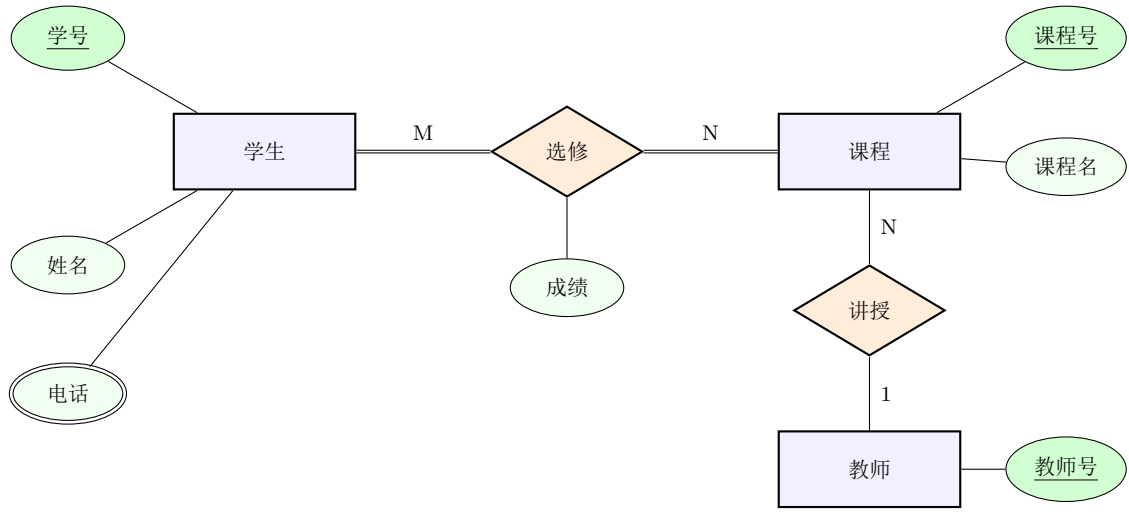


图 5.3: 一个 ER 示例：学生与课程为 $M:N$ 的「选修」（含属性「成绩」，双方全参与故用双线），教师与课程为 $1:N$ 的「讲授」

5.4 ER 到关系模式的转换

把 ER 图转换为关系模式（一组带主键、外键的表）遵循一组确定规则，汇总见表 5.5。转换的目标是**无损**：从 ER 语义能恢复出的信息，在关系模式中一样不能丢失。

表 5.5: ER 构件到关系模式的转换规则

ER 构件	转换结果	主键
强实体集	一张表，属性即列	实体集的主键
弱实体集	一张表	标识实体主键 + 自身部分键
1:1 联系	外键放任一侧（加唯一约束）或合并成表	不单独建表
1:N 联系	把 1 端主键作为外键放入 N 端	不单独建表
$M:N$ 联系	新建中间表，加入双方主键与联系属性	双方主键的复合键
多值属性	单独建表，含所属实体主键与该属性	实体主键 + 该属性
三元（及以上）联系	新建联系表，含三方主键与联系属性	三方主键的复合键

5.4.1 一对一与一对多

1:1 与 1:N 都不新建表，而是把主键作为外键「下沉」到合适的一侧：1:1 可放任一侧并加唯一约束（若一侧全参与，则外键放这一侧，且不能为空）；1:N 必须把外键放在 N 端。图 5.4 展示了 1:N 的转换。

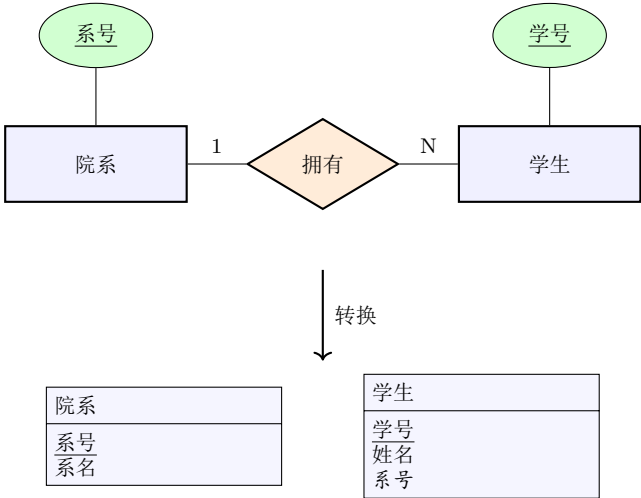


图 5.4: 1:N 转换：把 1 端主键「系号」作为外键放入 N 端「学生」

5.4.2 多对多

M:N 必须新建中间表（联系表），其主键是参与双方主键的复合主键，联系自身的属性也迁入该表。图 5.5 给出了转换，注意「成绩」找不到别的归属，只能随中间表保存。

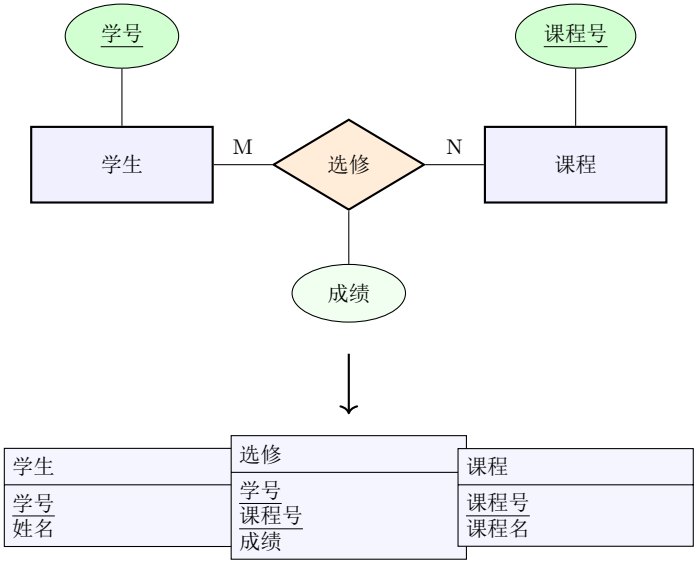


图 5.5: M:N 转换：新建中间表「选修」，主键为双方主键的复合键，联系属性一并迁入

5.4.3 弱实体、多值属性与三元联系

弱实体建表时，主键必须是「标识实体主键 + 部分键」的复合键，且标识实体主键同时是外键。例如订单项 (订单号, 行号, 数量)，其中订单号既是主键的一部分，又是引用订单的外键。

多值属性单独建表，键为「实体主键 + 该属性」。例如学生有多个电话：学生电话 (学号, 电话)。若省略这张表、把电话塞进「学生」的一列，就会在同一单元格里出现多个值，既违反 1NF，也无法按单个电话查询。

三元联系新建一张联系表，主键为三个参与实体主键的复合键，联系属性放入此表。例如供应 (供应商号, 项目号, 零件号, 供应量)。

5.4.4 转换实例

考虑一个图书馆借阅场景：读者借阅图书，每条借阅记录有借书日期与应还日期；一位读者可借多本书，一本书也可被多次借阅；每本书属于一个出版社。

ER 中的构件有：实体集读者（读者号、姓名）、图书（书号、书名）、出版社（社号、社名）；联系借阅（读者 $M:N$ 图书，属性借书日期、应还日期）、出版（出版社 $1:N$ 图书）。按规则转换得到：

$$\begin{aligned} &\text{读者}(\underline{\text{读者号}}, \text{姓名}), \quad \text{图书}(\underline{\text{书号}}, \text{书名}, \text{社号}), \\ &\text{出版社}(\underline{\text{社号}}, \text{社名}), \quad \text{借阅}(\underline{\text{读者号}}, \underline{\text{书号}}, \text{借书日期}, \text{应还日期}). \end{aligned} \quad (5.1)$$

其中「出版」是 $1:N$ ，外键社号下沉到 N 端「图书」；「借阅」是 $M:N$ ，单独建表并以 {读者号, 书号} 为复合主键，借书日期与应还日期随表迁入。至此，ER 语义被完整、无损地映射为关系模式。

5.5 函数依赖与范式

ER 建模保证了「有哪些表、哪些列、哪些外键」，但并不能保证「没有冗余」。判断一张表设计得好不好，需要函数依赖与范式理论。

5.5.1 函数依赖

函数依赖 (Functional Dependency, FD) $X \rightarrow Y$ 表示：在任何合法关系中，只要两个元组在 X 上相等，它们在 Y 上也必相等：

$$\forall t_1, t_2: \quad t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]. \quad (5.2)$$

直观上， $X \rightarrow Y$ 意味着「 X 唯一决定了 Y 」。若候选键为 K ，则 $K \rightarrow$ 所有属性；反过来，能决定所有属性的最小 X 就是候选键。FD 还满足如下推理规则 (Armstrong 公理)：自反 ($Y \subseteq X \Rightarrow X \rightarrow Y$)、增广 ($X \rightarrow Y \Rightarrow XZ \rightarrow YZ$)、传递 ($X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$)。

两个特别有害的依赖是：

- **部分依赖**： $X \rightarrow Y$ 且存在 X 的真子集 X' 使 $X' \rightarrow Y$ ，即 Y 只依赖复合键的一部分。例如 {学号, 课程号} $\rightarrow$ 姓名，而学号 $\rightarrow$ 姓名已成立，故姓名**部分依赖**于该复合键。
- **传递依赖**： $X \rightarrow Y, Y \rightarrow Z$ 且 $Y \not\rightarrow X$ 。例如学号 $\rightarrow$ 系名、系名 $\rightarrow$ 系主任，故系主任**传递依赖**于学号。

部分依赖与传递依赖会造成**更新异常**：同一事实在多个元组重复存储，插入、删除、修改时容易产生不一致。范式就是用来逐步消除这些依赖的。

5.5.2 范式的定义

范式的定义与所消除的问题见表 5.6。1NF 是基础；2NF 消除非主属性对候选键的部分依赖；3NF 进一步消除传递依赖；BCNF 则把约束加强到「每个决定因素都必须是超键」。

表 5.6: 范式的定义与所消除的异常		
范式	条件	消除的问题
1NF	所有属性都是原子的（不可再分）	多值属性、重复组
2NF	1NF，且每个非主属性完全依赖于候选键	非主属性对候选键的部分依赖
3NF	2NF，且非主属性不传递依赖于候选键	非主属性对候选键的传递依赖
BCNF	每个非平凡 $X \rightarrow A$ 的决定因素 X 都是超键	主属性之间的依赖

用依赖表示，若 F^+ 为函数依赖闭包：

$$3NF : \quad \forall X \rightarrow A \in F^+, \quad X \text{ 是超键} \vee A \text{ 是主属性}; \tag{5.3}$$

$$BCNF : \quad \forall X \rightarrow A \in F^+, A \notin X, \quad X \text{ 是超键}. \tag{5.4}$$

3NF 与 BCNF 的区别在于：3NF 允许「决定因素不是超键，但被决定的是主属性」这一例外，BCNF 则不允许。因此 BCNF 比 3NF 更严格。代价是，把一个模式分解到 BCNF 可能**损失函数依赖保持性**（分解后某些 FD 无法在单张表上检查），而 3NF 总能在**无损连接的同时保持依赖**。工程上常用 3NF 作为折中，只有在没有额外代价时才进一步追求 BCNF。

5.5.3 从非规范化到 3NF：一个完整分解

下面给出规范化分解的完整例题。

例题：某教学管理把选课信息记录为一张大表（非规范化）：

表 5.7: 未规范化的选课关系（存在冗余与异常）						
学号	姓名	系名	系主任	课程号	课程名	成绩
S1	张三	计算机	王五	C1	数据库	90
S1	张三	计算机	王五	C2	网络	85
S2	李四	计算机	王五	C1	数据库	78
S3	赵六	电子	孙七	C3	电路	88

设其函数依赖集为：

$$\begin{aligned} &\text{学号} \rightarrow \text{姓名, 系名}; \quad \text{系名} \rightarrow \text{系主任}; \\ &\text{课程号} \rightarrow \text{课程名}; \quad \{\text{学号, 课程号}\} \rightarrow \text{成绩}. \end{aligned} \tag{5.5}$$

候选键为 {学号, 课程号}，非主属性为姓名、系名、系主任、课程名、成绩。

第一步：满足 1NF。假定所有列都已原子化，则本表已是 1NF。若「电话」这类多值属性混在其中，应先拆出单独的表。

第二步：分解到 2NF。检查非主属性对候选键 {学号, 课程号} 的依赖：姓名、系名只依赖学号，课程名只依赖课程号，二者都是**部分依赖**，必须消除。按依赖的左部拆表：

学生(学号, 姓名, 系名, 系主任), 课程(课程号, 课程名), 选课(学号, 课程号, 成绩). (5.6)

此时每张表都满足 2NF：选课的非主属性成绩完全依赖整个键，另两张表的主键都是单属性，不存在部分依赖。

第三步：分解到 3NF。在「学生」中仍有传递依赖学号 $\rightarrow$ 系名、系名 $\rightarrow$ 系主任且系名 $\nrightarrow$ 学号，故系主任传递依赖于学号，必须把「系」独立出去：

学生(学号, 姓名, 系名), 系(系名, 系主任), 课程(课程号, 课程名), 选课(学号, 课程号, 成绩). (5.7)

四张表均满足 3NF，且该分解无损连接并保持全部函数依赖。原表中的冗余（系主任在多个元组重复）与更新异常随之消失：改系主任只需改「系」表一处；删掉最后一次选课也不会误删系主任信息。

5.6 本章小结

本章围绕「语义 $\rightarrow$ 结构 $\rightarrow$ 质量」三条线索展开：

1. **ER 构件**：实体集（矩形）、属性（椭圆，分简单/复合/多值/派生）、键（主键、候选键、外键、部分键）、弱实体（双矩形）。
2. **联系**：度（一元/二元/三元）、基数（1:1、1:N、M:N）、参与约束（双线为全部）、属性化联系。
3. **转换规则**：强实体直接建表；1:1、1:N 用外键下沉；M:N、三元联系、多值属性建新表并构造复合主键；弱实体主键含标识实体主键。
4. **质量**：函数依赖刻画「谁能决定谁」；1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF 依次消除部分依赖、传递依赖与主属性间的异常，规范化通过分解实现。

5.7 常见误区

- **把 M:N 联系当成普通字段**：忘记建中间表，或把「课程号」塞进学生表的一列，导致多值单元格，违反 1NF。
- **给 1:N 联系错建中间表**：把外键放错端（放到 1 端），既冗余又无法表达「一个学生只属于一个系」。
- **弱实体主键漏掉标识实体主键**：只用「行号」作主键，不同订单的行号会冲突；正确主键是 {订单号, 行号}。
- **主键与候选键、外键混淆**：候选键可以多个，主键是其中被选定的一个；外键是引用约束而非本表的唯一性约束，且可能为空。
- **基数与参与约束混淆**：二者正交——基数回答「最多几个」，参与约束回答「能不能为零」。
- **混用实体与联系**：把「选课」当成实体又当成联系，造成重复建模与语义冲突。

-
- **认为 BCNF 一定优于 3NF：**BCNF 分解可能丢失函数依赖保持性，工程上常以 3NF 为折中。
 - **冗余存储派生属性：**如把「年龄」存入表而不随出生日期更新，日积月累造成数据不一致。
 - **三元联系拆成三个二元联系：**会丢失「三者同时组合」的语义，例如无法表达「某供应商只对某项目的某零件有供货关系」。

第六章 设计原则与模式

良好的设计在「可扩展」与「可维护」之间取得平衡：SOLID 原则指导类的职责划分，设计模式沉淀可复用的协作经验，软件架构决定系统的整体骨架。

需求工程回答「做什么」，软件设计回答「怎么做」。设计把需求模型逐步转化为可编码、可验证的蓝图，是衔接分析与实现的关键桥梁。本章先讲设计过程与基本目标，再讲指导类设计的 SOLID 原则，然后给出可复用的设计模式，接着讨论决定系统整体骨架的体系结构风格，最后用耦合与内聚两类度量把「设计好坏」量化。

6.1 设计过程与目标

6.1.1 从需求到实现

软件设计通常分四类活动，自顶向下、由粗到精地展开：

表 6.1: 软件设计的四类活动

设计活动	关注问题	典型产物
体系结构设计	系统由哪些子系统组成、如何交互、技术选型	架构图、分层/MVC/微服务方案
数据设计	数据如何组织、存储、持久化	ER 图、表结构、缓存策略
接口设计	模块之间「约定什么」	接口签名、消息格式、API 契约
组件设计	每个模块内部「怎么实现」	类图、算法、状态机

四类活动并非严格串行：接口设计会反过来约束组件设计，体系结构也可能在组件细化时被修正。

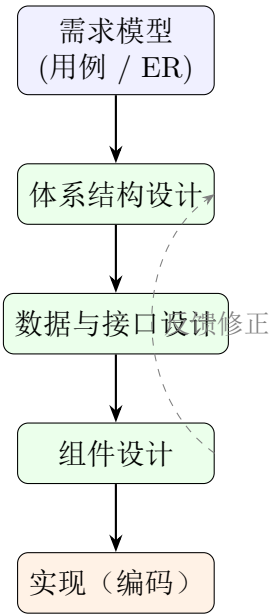


图 6.1: 需求到实现：设计逐层细化，并允许反馈修正

6.1.2 四条基本目标

- **抽象 (Abstraction):** 忽略与当前问题无关的细节，只保留本质特征。分出**过程抽象**（把「一组操作」看成一个动作）与**数据抽象**（把「一组数据及其操作」看成一个类型）。
- **模块化 (Modularization):** 把系统分解为可独立开发、测试、替换的模块。分而治之能降低单点复杂度，但分解过细又会抬高模块间的通信成本，存在最优粒度。
- **信息隐藏 (Information Hiding):** 模块只通过接口暴露「做什么」，把「怎么做」——数据结构、算法、易变细节——藏在内部。调用者只依赖契约，实现可自由替换。

- **逐步求精 (Stepwise Refinement):** 自顶向下、由粗到精地补充细节，每一步都在前一步的抽象之上展开，直到足以编码。

表 6.2: 抽象与信息隐藏的对比

概念	侧重点	在图书馆系统中的体现
抽象	保留本质、丢弃细节	把「借书」定义为接口方法 <code>borrow()</code> ，不关心内部如何校验
信息隐藏	对外只露接口、对内封装实现	把「逾期天数计算规则」藏在 <code>Loan</code> 内部，规则变化不外溢

6.2 软件设计原则

面向对象设计的基本准则是「高内聚、低耦合」。SOLID 是这一准则的五个可操作推论，目标是让系统在需求变化时「改动局部、稳定整体」。

表 6.3: SOLID 五原则：含义与反例改进

原则	含义	反例 → 改进
单一职责 (SRP)	一个类只有一个引起变化的原因	上帝类 → 按职责拆分
开闭 (OCP)	对扩展开放、对修改关闭	每加类型就改 <code>switch</code> → 引入多态/策略
里氏替换 (LSP)	子类型可替换父类型而不破坏契约	子类抛异常、加强前置条件 → 保持契约
接口隔离 (ISP)	客户端不应依赖不需要的接口	胖接口 → 拆成小而专的接口
依赖倒置 (DIP)	高层与低层都依赖抽象	高层 <code>new</code> 具体实现 → 依赖接口注入

6.2.1 单一职责原则 (SRP)

定义：一个类应当只有一个引起它变化的原因，即只承担一项职责。

反例：`Invoice` 类同时负责「计算金额」「格式化输出」「写入数据库」。财务规则、打印格式、存储介质任一变化都要改这个类，且三种变更相互干扰。

改进：拆分为 `Invoice`（金额计算）、`InvoicePrinter`（格式化）、`InvoiceRepository`（持久化）。每个类只随一类需求变化。

6.2.2 开闭原则 (OCP)

定义：软件实体应对扩展开放、对修改关闭。新增功能应尽量通过「增加代码」而非「修改已有代码」完成。

反例——每次新增折扣类型都要改 `switch`:

```
1 // 反例：每加一种会员类型就要修改这个方法
2 double discount(Member m) {
3     switch (m.getType()) {
4         case "NORMAL": return 1.0;
5         case "VIP":     return 0.9;
6         // 新增类型 -> 修改此处
7     }
```

```

8  }
9
10 // 改进：把算法抽象为接口，新增类型只需新增一个类
11 interface PricingPolicy { double discount(Member m); }
12 class VipPricing implements PricingPolicy {
13     public double discount(Member m) { return 0.9; }
14 }

```

Listing 6.1: OCP 反例与改进

改进：引入多态或策略模式，把易变的分支逻辑收敛到可扩展的抽象之后。

6.2.3 里氏替换原则（LSP）

定义：所有引用基类的地方必须能透明地使用其子类对象，且不改变程序的可期望行为。

反例：Square 继承 Rectangle 并覆写 setWidth 同时修改宽和高。调用者按矩形语义先设宽再设高，得到的面积与预期不符。子类**加强了前置/后置约束**，破坏了基类契约。

改进：不要用「正方形是矩形」的数学直觉做继承；让二者共享一个 Shape 接口，各自实现 area()。

6.2.4 接口隔离原则（ISP）

定义：客户端不应被强迫依赖它不使用的接口，应把「胖接口」拆成多个小而专的接口。

反例：一个 Worker 接口含 work()、eat()、sleep()，机器人工人被迫实现 eat()、sleep() 并抛出异常。

改进：拆成 Workable 与 Feedable，机器人工人只实现 Workable。

6.2.5 依赖倒置原则（DIP）

定义：高层模块不应依赖低层模块，二者都应依赖抽象；抽象不应依赖细节，细节应依赖抽象。

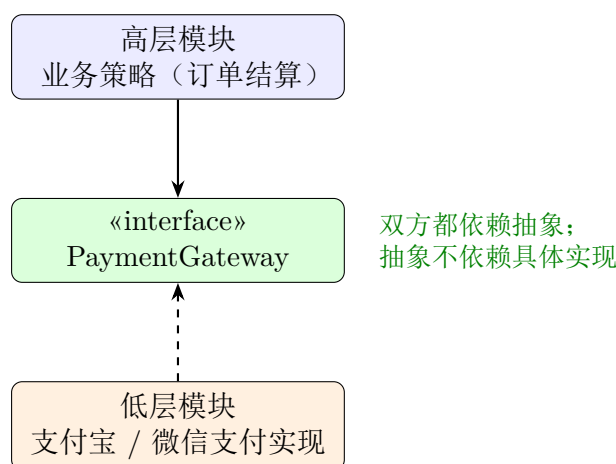


图 6.2: 依赖倒置：高层与低层都依赖抽象，抽象不依赖细节

反例：结算服务直接 new AlipayGateway(), 换支付渠道就必须改高层代码。**改进**：高层持有 PaymentGateway 接口，由外部注入具体实现，既方便替换，也便于在测试中注入替身（mock）。

6.3 设计模式

设计模式是对「特定上下文中反复出现的设计问题」的可复用解决方案。它给出的是结构化经验，而非可直接拷贝的代码。

6.3.1 模式分类

表 6.4: 设计模式三大分类

类型	关注点	代表模式
创建型 (Creational)	对象创建机制，解耦创建与使用	单例、工厂方法、抽象工厂、建造者、原型
结构型 (Structural)	类与对象的组合，组装更大结构	适配器、装饰器、代理、外观、组合
行为型 (Behavioral)	对象间职责分配与通信	观察者、策略、命令、状态、模板方法

6.3.2 代表模式

表 6.5: 代表设计模式的意图、结构与适用场景

模式	意图	结构要点	适用
单例 (Singleton)	保证仅有一个实例并提供全局访问点	私有构造 + 静态实例 + 静态获取方法	全局配置、连接池、日志器
工厂方法 (Factory Method)	定义创建接口，由子类决定实例化哪个类	Creator 声明工厂方法，ConcreteCreator 返回产品	框架扩展点、解耦创建与使用
观察者 (Observer)	一对多依赖，主题变化时自动通知所有观察者	Subject 维护观察者列表，状态变化即遍历通知	事件通知、GUI 事件、消息订阅
策略 (Strategy)	封装可互换的算法，使其可独立变化	Context 持有 Strategy 接口，运行时替换实现	多种计费/排序/压缩算法切换
装饰器 (Decorator)	动态给对象添加职责，比继承更灵活	装饰器实现同一接口并持有被装饰对象	流式 I/O、动态叠加功能

策略模式

意图：把一族可互换算法各自封装成类，让它们可以互相替换，且算法的变化独立于使用它的客户端。

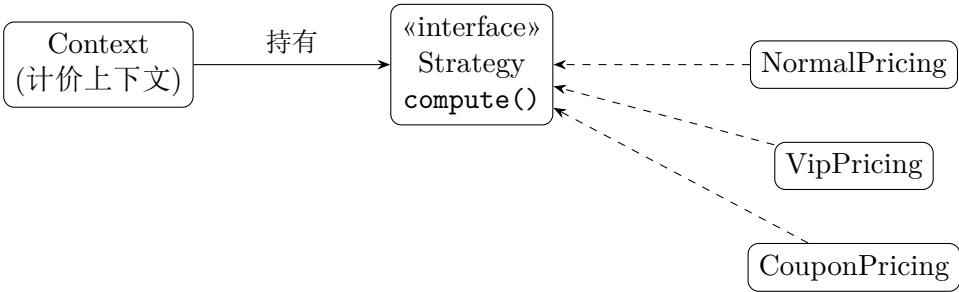


图 6.3: 策略模式：上下文持有抽象策略，运行时替换具体算法

适用：结算有会员价、优惠券价等多种算法，且未来会持续新增。若用 `if-else` 堆叠，每次新增都要改动上下文，违反 OCP；用策略则只需新增一个实现类。

观察者模式

意图：定义对象间的一对多依赖，当主题状态改变时，所有依赖它的观察者都会自动收到通知并更新。

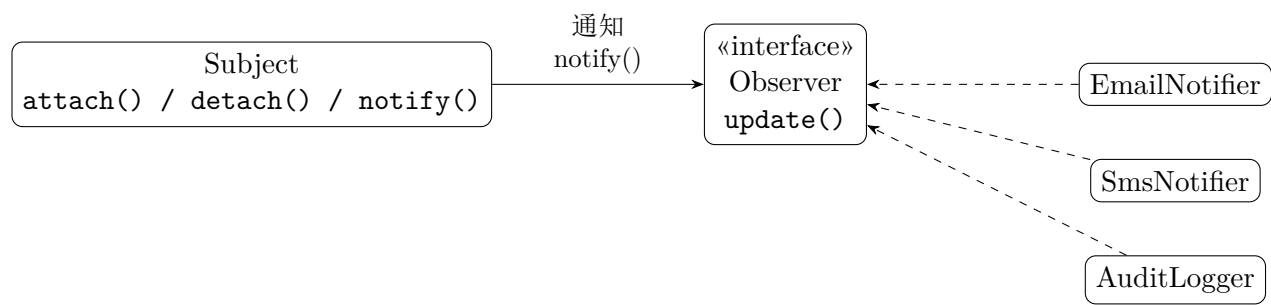


图 6.4: 观察者模式：主题在状态变化时遍历通知观察者列表

适用：图书借阅系统中，「借阅成功」这一主题事件需要触发邮件、短信、审计日志三种响应，且响应方会持续增加。观察者让主题只依赖抽象，新增响应方无需改动主题。

6.4 软件体系结构风格

体系结构风格规定系统「整体如何组织」：子系统如何划分、如何交互、数据与控制如何流动。选型的核心是**权衡**，没有普适最优。

表 6.6: 常见软件体系结构风格对比

风格	特点	优点	代价
分层 (Layered)	上层调用下层，层间通过接口隔离	结构清晰、易测试、关注点分离	调用穿透、性能损耗
MVC	模型、视图、控制器分离，视图随模型更新	UI 与业务解耦、多视图复用	控制器易膨胀
微服务 (Microservices)	按业务边界拆为独立部署的服务	独立伸缩、技术异构、故障隔离	分布式与运维复杂
事件驱动 (Event-driven)	组件通过事件异步通信、发布-订阅	松耦合、可扩展、异步削峰	最终一致、调试困难
仓库 (Repository)	所有组件共享一个中心数据仓库	数据一致、便于集成	中心仓库成为瓶颈与单点

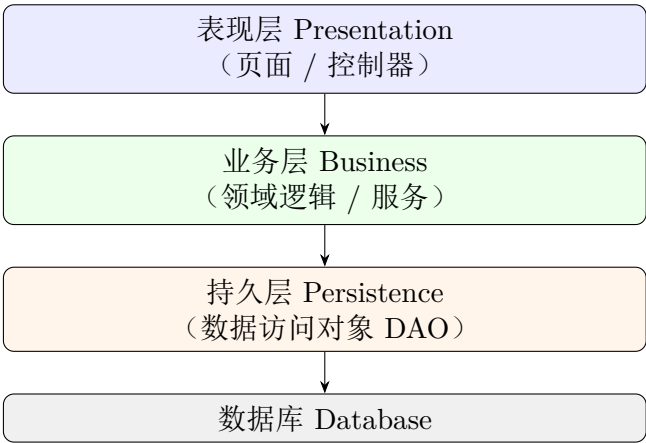


图 6.5: 分层架构：关注点自上而下逐层分离，每层仅调用相邻下层

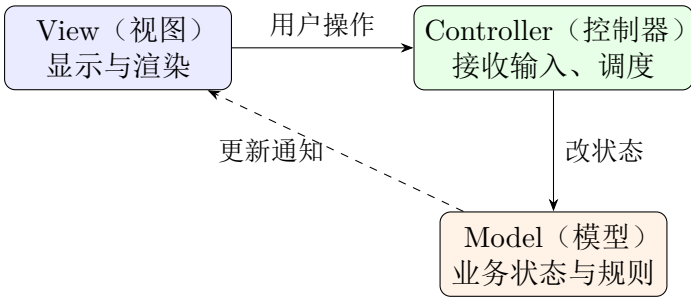


图 6.6: MVC：控制器处理输入、模型保存状态、视图订阅模型

选型建议：分层适合关注点分离的常规业务系统；MVC 适合交互式 UI；微服务适合需要独立伸缩与团队自治的大型系统；事件驱动适合高并发、松耦合的集成场景；仓库风格适合以数据一致性为核心的批处理系统。

6.5 设计度量

「高内聚、低耦合」是设计的核心目标，可以用两个方向相反的度量刻画**模块独立性**：

- **内聚 (Cohesion)**：模块内部各元素彼此关联的紧密程度。内聚**越高越好**。
- **耦合 (Coupling)**：模块之间相互依赖的紧密程度。耦合**越低越好**。

表 6.7: 内聚的七个等级（由高到低）

等级	含义	评价
功能内聚	所有元素共同完成 单一 功能	最好
顺序内聚	元素输出是下一元素的输入	好
通信内聚	元素操作 同一 数据集	较好
过程内聚	元素按特定执行顺序组合	中
时间内聚	元素在同一时间段内执行	较差
逻辑内聚	元素由参数选择执行其一	差
偶然内聚	元素之间没有实质关系	最差

表 6.8: 耦合的六个等级（由低到高）

等级	含义	评价
数据耦合	仅通过参数传递简单数据	最好
标记耦合	传递整个数据结构，但只用到其中一部分	较好
控制耦合	传递控制标志以指挥对方行为	中
外部耦合	依赖外部环境/协议	较差
公共耦合	多个模块共享全局数据	差
内容耦合	一个模块直接访问另一模块内部	最差

设计的目标是**提高内聚、降低耦合**：把强相关的逻辑收进同一模块（功能内聚），把模块之间的交互退化为最小必要的数据耦合，并用接口与信息隐藏隔离易变实现。

6.6 本章小结与常见误区

本章从设计过程出发，建立了「过程 → 原则 → 模式 → 架构 → 度量」的主线：设计把需求细化为可实现的蓝图；SOLID 指导类的职责划分；设计模式沉淀可复用的协作经验；体系结构决定系统骨架；耦合与内聚把设计质量量化。

常见误区：

- **为模式而模式**：小系统硬套抽象工厂、观察者，反而增加理解成本。模式应服务于变化点。
- **把继承当复用**：为复用代码滥用继承，容易触犯 LSP，且父类变动冲击所有子类。优先组合而非继承。
- **误解「对修改关闭」**：OCP 不是禁止修改任何代码，而是把变化隔离到可扩展点，让核心逻辑免于频繁改动。
- **把分层当万能**：层层穿透调用会带来性能损耗；分层过细还会制造大量无意义的转发代码。
- **只谈概念不看度量**：设计优劣要落到耦合与内聚上，能用度量解释「为什么这样拆」才是有效设计。

第七章 软件测试

软件测试通过「执行程序并观察结果」来暴露缺陷，是质量保证的最后一道防线。本章覆盖测试层次与 V 模型、黑盒与白盒测试方法，以及测试用例设计与回归测试。

7.1 测试目标与原则

7.1.1 测试的定义与目标

软件测试（Software Testing）是使用人工或自动手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的需求，或是弄清预期结果与实际结果之间的差别。Myers 给出了更强调「找错」的定义：测试是为了发现错误而执行程序的过程；一个好的测试用例是那些能够发现尚未发现的错误的用例，而一次「成功」的测试正是找出了新缺陷的测试。

测试服务于两个目标：

- **验证（Verification）**：回答「我们是否正确地构造了产品？」（Are we building the product right?），即检查程序是否符合规格说明与设计文档。
- **确认（Validation）**：回答「我们是否构造了正确的产品？」（Are we building the right product?），即检查系统是否真正满足用户的业务需求。

因此，测试不只是「运行程序」，更是一个贯穿需求到维护的信息收集与质量评估活动：它既发现缺陷，也为项目决策提供关于质量与风险的信息。

7.1.2 测试与调试的区别

初学者常把测试与调试混为一谈，但二者在目的、起点与执行者上都不同，见表 7.1。

表 7.1: 测试与调试的区别

维度	测试（Testing）	调试（Debugging）
目的	发现缺陷是否存在	定位并修复已发现的缺陷
起点	已知需求/规格，未知缺陷	已知缺陷（失败用例）
方法	有计划地设计用例并执行	推理、假设检验、逐段排查
活动	设计、执行、比较、报告	复现、定位、修改、验证
结束标志	按计划完成、覆盖率达标	缺陷被修复、失败用例通过
执行者	独立测试人员	开发人员
特征	可计划、可重复、可度量	依赖经验与直觉，难预测

可见：测试是「找病」，调试是「治病」。测试提供症状（失败现象），调试据此定位病根并给出处方。

7.1.3 测试的基本原则

- **尽早测试（Test Early）**：缺陷发现得越早，修复代价越低。缺陷的破坏性随时间放大，见表 7.2。
- **独立测试**：开发人员应避免测试自己的程序（心理上不愿否定自己的成果），关键系统应由独立测试团队或第三方执行，测试用例也应独立设计。
- **缺陷聚集（Defect Clustering）**：经验表明约 80% 的缺陷集中在约 20% 的模块中（二八法则）。测试资源应优先投向高风险、高复杂度模块。
- **经济性（Economies）**：穷尽测试在数学上不可能（输入组合爆炸），必须围绕风险与成本选择「性价比最高」的用例集合。

- **杀虫剂悖论 (Pesticide Paradox):** 同一组用例反复使用后，其发现新缺陷的能力会下降，需定期评审并更新用例。
- **无错谬误:** 即使程序没有错误，如果需求本身有误，它仍是无用的；测试不能替代需求验证。

表 7.2: 缺陷在不同阶段被发现时的相对修复成本

发现阶段	相对修复成本	说明
需求/设计评审	1	早期静态检查，代价最低
编码	5	修改局部代码
单元/集成测试	10	需回改设计与代码
系统测试	20	影响面扩散，需回归
验收/发布后	50 ~ 100	可能涉及版本回退与用户损失

Dijkstra 有一句名言概括了测试的极限：测试能证明缺陷的存在，却无法证明缺陷的不存在。因此测试是一类「抽样」活动，其结论只能降低风险，而不能给出「零缺陷」的保证。

7.2 测试层次与 V 模型

7.2.1 四个测试层次

软件测试按「由小到大、由内到外」分为四个层次：先保证单元正确，再保证组装正确，最后验证系统与业务目标，见表 7.3。

表 7.3: 测试层次的目标、对象与执行者

层次	测试对象	目标	执行者
单元测试	函数/方法/类	验证模块内部逻辑与分支	开发人员
集成测试	模块间的接口	验证接口、数据传递与协作	开发/测试人员
系统测试	完整系统	验证功能与非功能需求	独立测试团队
验收测试	交付给用户的系统	确认是否满足业务需求	用户/客户

7.2.2 V 模型

V 模型把开发过程与测试过程对应起来：左边自顶向下逐层细化（需求 → 编码），右边自底向上逐层验证（单元 → 验收）。关键思想是每一层的测试计划应在对应的开发阶段就制定：需求阶段写验收/系统测试计划，设计阶段写集成测试计划，编码阶段写单元测试计划，见图 7.1。

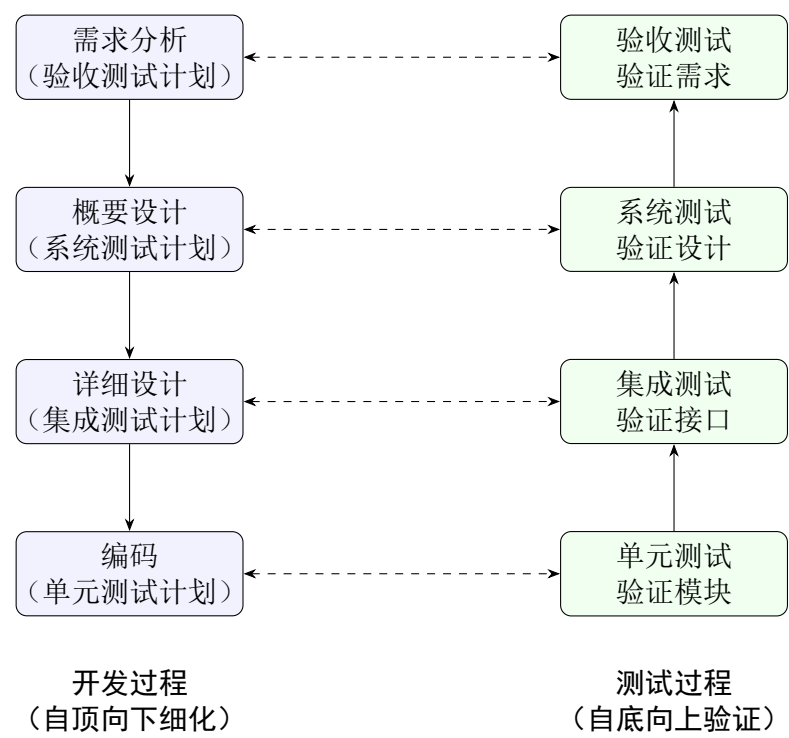


图 7.1: V 模型：左侧开发阶段与右侧测试阶段一一对应

7.2.3 集成测试策略、桩与驱动

集成测试把已通过单元测试的模块按设计组装起来，重点验证接口。由于被集成时某些下层或上层模块尚未就绪，需要用替身模块占位：

- 桩 (Stub)：模拟被测模块所调用的下层模块，返回预定的（可能是伪造的）结果。
- 驱动 (Driver)：模拟调用被测模块的上层模块，负责提供输入、触发调用并输出结果。

三种常见集成策略见表 7.4。

表 7.4: 集成测试的三种策略			
策略	集成方向	优点	缺点
自顶向下	从主控模块向下	只需桩；早期验证主控逻辑，利于演示	需大量桩；底层关键模块晚测试
自底向上	从底层模块向上	只需驱动；早期验证底层关键模块	主控逻辑与整体框架晚验证，不能早期演示
三明治	上下同时、向中间汇合	兼顾两者；适合中间层最关键的场景	桩与驱动都需，成本较高

自底向上集成只需编写驱动，其过程如算法 1 所示。

Algorithm 1 自底向上集成测试

```
1: procedure BOTTOMUPINTEGRATION(模块集合 M)
2:   按调用层次把 M 划分为若干层
3:   for 每个底层模块 m do
4:     为 m 编写驱动模块 Dm
5:     用 Dm 独立测试 m
6:   end for
7:   while M 中仍有未集成的模块 do
8:     选取其所有下层模块均已测试通过的模块 p
9:     用驱动替代尚未就绪的上层模块，测试 p 与其下层模块的接口
10:    将测试通过的 p 加入已集成集合
11:  end while
12: end procedure
```

7.3 黑盒测试方法

黑盒测试（功能测试）只关心「输入 → 输出」，不考虑程序内部结构，依据规格说明设计用例。常用方法有等价类划分、边界值分析、因果图/判定表和错误推测。

7.3.1 等价类划分

等价类划分把输入域划分为若干子集，使同一子集中的任意输入对「发现缺陷」具有相同效果，于是每类只需取一个代表值。等价类分为有效等价类（合法输入）与无效等价类（非法输入）。

- 有效等价类：判断程序是否实现了规定的功能。
- 无效等价类：判断程序对非法输入是否正确处理（报错、容错）。

设计规则：为每个有效等价类设计一个用例；为每个无效等价类单独设计一个用例（一次只覆盖一个无效类，避免相互掩盖）。

例题：某输入框要求年龄 x 为 $[1, 100]$ 的整数。等价类划分见表 7.5。

表 7.5: 年龄输入的等价类划分（要求 $x \in [1, 100]$ 的整数）

类别	编号	描述	代表值
有效	E1	$1 \leq x \leq 100$ 的整数	50
无效	E2	$x < 1$ （低于下界）	0
无效	E3	$x > 100$ （高于上界）	101
无效	E4	非整数（含小数、非数字）	"abc"
无效	E5	空输入	（空串）

由表 7.5 可得 5 个用例：E1 期待「接受」，E2-E5 期待「拒绝并给出相应提示」。

7.3.2 边界值分析

缺陷大多藏在边界，因为开发者常把条件写成 $</>$ ，处理「等于边界」时易出错。边界值分析在等价类的基础上，取每类的边界及其邻点作为用例。对区间 $[1,100]$ ，应取 $\min - 1, \min, \min + 1, \max - 1, \max, \max + 1$ 共 6 个边界值，见表 7.6。

表 7.6: 边界值分析示例（输入范围 $[1,100]$ ）

边界点	取值	预期结果
$\min - 1$	0	无效（低于下界）
$\min$	1	有效
$\min + 1$	2	有效
$\max - 1$	99	有效
$\max$	100	有效
$\max + 1$	101	无效（高于上界）

边界值 vs. 等价类：等价类划分关注「类」，边界值分析关注「点」。工程上二者常结合使用——先用等价类确定输入域，再在每类边界取点。

7.3.3 因果图与判定表

当输入之间存在**组合与约束**时，逐个条件孤立测试会遗漏组合缺陷。因果图把「原因（输入条件）」与「结果（输出动作）」用图形表达，再用**判定表**把逻辑组合展开为规则。

判定表由四部分组成：**条件桩**（所有条件）、**动作桩**（所有动作）、**条件项**（各条件的取值）、**动作项**（各规则下执行的动作）。条件取值为 T （真）、 F （假）、 $-$ （无关，可化简合并）。

例题：登录功能的判定表见表 7.7，其中每条竖列称为一条规则。

表 7.7: 登录功能的判定表

条件 / 动作	规则 R1	规则 R2	规则 R3
条件桩：用户已注册 C_1	T	F	T
条件桩：密码正确 C_2	T	$-$	F
动作桩：登录成功 A_1	✓		
动作桩：提示「用户不存在」 A_2		✓	
动作桩：提示「密码错误」 A_3			✓

因果图中常用约束记号见（行内说明）：**异（E）**表示至多一个原因为真；**或（I）**表示至少一个为真；**唯一（O）**表示恰好一个为真；**要求（R）**表示若 a 为真则 b 必须为真；**屏蔽（M）**表示 a 为真时 b 必为假。判定表由「条件桩 $\times$ 动作桩」的逻辑组合构成，是因果图的表格化形式，也是自动生成用例的依据。

7.3.4 错误推测

错误推测依靠测试人员的经验与直觉，列出程序可能出错的情形并据此设计用例。常见猜测对象包括：空值/零值、非法输入、越界数据、重复数据、并发冲突、以及历史上同类模块易错之处。错误推测效率高但依赖个人经验，通常作为等价类、边界值等方法之外的**补充**。

7.4 白盒测试与逻辑覆盖

白盒测试（结构测试）依据程序内部逻辑设计用例，以覆盖率衡量测试的充分程度。逻辑覆盖准则由弱到强依次为：语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖、路径覆盖。

7.4.1 被测程序示例

考虑如下函数：判定 $D = (a > 0) \ \&\& \ (b > 0)$ 由两个原子条件 $C_1 : a > 0$ 、 $C_2 : b > 0$ 组成。

```
1 int f(int a, int b) {
2     int x = 0;
3     if (a > 0 && b > 0) {    /* 判定 D = C1 && C2 */
4         x = a + b;           /* 语句 S */
5     }
6     return x;
7 }
```

7.4.2 六种覆盖准则

- 语句覆盖：每条可执行语句至少执行一次。
- 判定覆盖（分支覆盖）：每个判定的真、假分支至少各执行一次。
- 条件覆盖：每个原子条件的真、假至少各取一次。
- 判定/条件覆盖：同时满足判定覆盖与条件覆盖。
- 条件组合覆盖：每个判定中各原子条件的所有真假组合至少出现一次。
- 路径覆盖：源程序中每条可能的路径至少执行一次，最强但路径数可能指数增长。

针对上例，各准则的最少用例数与代表性用例见表 7.8。

表 7.8: 逻辑覆盖准则的最少用例数与强弱关系（示例 $D = (a > 0) \ \&\& \ (b > 0)$ ）

覆盖准则	最少用例数	代表性用例 (a, b)
语句覆盖	1	$(2, 2)$
判定覆盖	2	$(2, 2)$ 、 $(-1, 2)$
条件覆盖	2	$(2, 2)$ 、 $(-1, -1)$
判定/条件覆盖	2	$(2, 2)$ 、 $(-1, -1)$
条件组合覆盖	4	$(2, 2)$ 、 $(2, -1)$ 、 $(-1, 2)$ 、 $(-1, -1)$
路径覆盖	2	$(2, 2)$ 、 $(-1, 2)$

强弱关系：语句覆盖最弱；满足判定覆盖一定满足语句覆盖（判定覆盖 $\supseteq$ 语句覆盖），但语句覆盖不一定满足判定覆盖。条件覆盖不蕴含判定覆盖（满足了各条件真假，判定可能仍只取一个分支），反之判定覆盖也不蕴含条件覆盖，因此引入判定/条件覆盖同时保证二者。条件组合覆盖蕴含判定/条件覆盖，是最强的条件类准则。路径覆盖最强，但独立路径数随分支爆炸，工程上常用基于圈复杂度 $V(G) = E - N + 2$ 的基本路径覆盖加以控制。

7.4.3 基本路径测试

圈复杂度（Cyclomatic Complexity） $V(G) = E - N + 2$ （ E 为控制流图边数， N 为结点数）既是独立路径数下界，也是程序复杂度的度量。基本路径测试以 $V(G)$ 为目标，构造一组线性独立的路径并为之设计用例（算法 2）。

Algorithm 2 基本路径测试用例设计

```
Require: 被测模块的控制流图  $G$  与模块接口
Ensure: 一组覆盖全部基本路径的测试用例
1: 由  $G$  计算  $V(G) = E - N + 2$ 
2: 选取一条「最典型」的路径作为基线路径
3: for  $i = 1$  to  $V(G)$  do
4:   通过翻转某个判定结点，导出第  $i$  条独立路径
5: end for
6: for 每条独立路径 do
7:   反推能沿该路径执行的输入数据，生成一个用例
8: end for
```

7.5 测试用例设计综合与回归测试

7.5.1 测试用例的要素与设计顺序

一个好的测试用例应包含四要素：前置条件、输入数据、操作步骤、预期输出，并且必须能唯一判定「通过/失败」。用例模板见表 7.9。

表 7.9: 测试用例模板示例（登录功能）

用例编号	前置条件	输入	预期输出
TC-01	用户已注册	正确账号 + 正确密码	登录成功，进入主页
TC-02	无	未注册账号 + 任意密码	提示「用户不存在」
TC-03	用户已注册	正确账号 + 错误密码	提示「密码错误」
TC-04	用户已注册	账号正确、密码为空	提示「密码不能为空」

黑盒与白盒各有所长，实践中应组合使用，设计顺序通常为：

- 先用等价类划分确定输入域并覆盖有效/无效类；
- 再用边界值分析补齐各类的边界点；
- 用因果图/判定表处理多条件组合；
- 用错误推测追加经验性用例；
- 最后用白盒覆盖准则检查分支/路径，补齐尚未覆盖的逻辑。

7.5.2 回归测试

修改代码（修缺陷、加功能、重构）后，必须重新运行测试以避免引入新缺陷或破坏既有功能，这一活动称为回归测试。常见策略见表 7.10。

表 7.10: 回归测试的常见策略		
策略	范围	说明
全量回归	全部用例	最安全但代价最高，适合发布前
选择回归	受影响模块及依赖	依据变更影响分析挑选用例
优先级回归	高风险/高优先级用例	先跑关键路径，尽早暴露问题

为降低回归成本，应尽可能自动化测试：把核心用例固化为可重复运行的脚本，并纳入持续集成（CI），使每次提交都自动触发回归。

7.6 本章小结

本章围绕「目标 → 层次 → 方法 → 设计」展开：

- **目标与原则：**测试兼有验证与确认双重目标，遵循尽早、独立、缺陷聚集、经济性、杀虫剂悖论等原则；测试与调试是「找病」与「治病」两种不同活动。
- **层次与 V 模型：**单元、集成、系统、验收由小到大；V 模型把开发与测试阶段一一对应，测试计划前移。集成时用桩（模拟下层）与驱动（模拟上层），可采用自顶向下、自底向上或三明治策略。
- **黑盒方法：**等价类划分、边界值分析、因果图/判定表、错误推测，从规格说明出发，只关注输入输出。
- **白盒方法：**语句、判定、条件、判定/条件、条件组合、路径覆盖由弱到强；以圈复杂度控制基本路径数量。
- **综合与回归：**用例需含前置条件、输入、步骤、预期输出；黑盒与白盒组合设计，修改后执行回归测试。

常见误区

表 7.11: 测试中的常见误区与正确认识	
常见误区	正确认识
测试就是调试	测试负责发现缺陷，调试负责定位与修复，二者执行者与目标不同
测试通过就说明没有缺陷	测试是抽样，只能证明缺陷存在，不能证明缺陷不存在
测试应在编码完成后才开始	测试计划与用例应随需求、设计同步制定，尽早暴露缺陷
覆盖率达到 100% 即质量达标	覆盖率只反映代码被执行，不反映逻辑正确；语句覆盖 100% 仍可能漏判
开发者测试自己的程序足够	独立性原则要求关键系统由独立人员或第三方测试
需要做到穷尽测试	输入组合爆炸，应基于风险与成本选择高性价比用例
用固定用例反复测试即可	杀虫剂悖论：用例需随系统演进而更新迭代

第八章 项目管理与质量

项目管理回答四个问题：要多久、花多少、有多少风险、质量如何度量。本章沿「估算-进度-风险-质量」四步展开，把成本模型、关键路径法、风险应对与质量度量落到可计算的公式与表格上。

8.1 规模与成本估算

8.1.1 代码行 LOC 与功能点 FP

代码行 (LOC, Lines of Code) 是最直观的规模度量，通常以千行 **KLOC** 为单位。它易于统计，但依赖语言与编程风格，且在设计完成前难以准确预估。

功能点 (FP, Function Points) 从「用户可见的功能」出发，与语言无关。先按复杂度给五类功能元素加权，得到未调整功能点 *UFP*：

表 8.1: 功能点五类元素及复杂度权重			
功能元素	简单	平均	复杂
外部输入 EI	3	4	6
外部输出 EO	4	5	7
外部查询 EQ	3	4	6
内部逻辑文件 ILF	7	10	15
外部接口文件 EIF	5	7	10

再用 14 项通用系统特征（数据通信、性能、重用性等）的复杂度 $F_i \in [0, 5]$ 做调整：

$$FP = UFP \times \left(0.65 + 0.01 \sum_{i=1}^{14} F_i \right) \tag{8.1}$$

其中调整因子取值范围为 0.65 ~ 1.35。功能点可在设计前估算，再换算成 KLOC 以套用成本模型。

8.1.2 COCOMO 模型

COCOMO (Constructive Cost Model) 把工作量表示为规模的幂函数。基本模型按项目类型取参数 a, b, c, d ：

表 8.2: COCOMO 基本模型参数			
项目类型	a	b	说明
组织型 (Organic)	2.4	1.05	小型、经验丰富、约束少
半独立型 (Semi-detached)	3.0	1.12	中等规模、混合经验
嵌入式 (Embedded)	3.6	1.20	硬约束、复杂接口

组织型对应的基本公式为

$$E = 2.4 \times (KLOC)^{1.05} \quad (\text{人月, person-months}) \tag{8.2}$$

$$D = 2.5 \times E^{0.38} \quad (\text{开发月数, months}) \tag{8.3}$$

$$\text{团队规模} = \frac{E}{D} \quad (\text{人}) \tag{8.4}$$

例如 $KLOC = 33$ （组织型）时： $E = 2.4 \times 33^{1.05} \approx 94.3$ 人月， $D = 2.5 \times 94.3^{0.38} \approx 14.1$ 月，团队规模约 6.7 人。可见工作量随规模**超线性**增长——把规模翻倍，工作量增长超过一倍，这正是「人月神话」的量化根源。

表 8.3: 常用成本估算方法对比

方法	优点	局限
代码行 LOC	直观、易统计	语言相关、需设计后
功能点 FP	与语言无关、早期可用	主观、需经验校准
COCOMO	公式化、可调参	依赖历史数据与规模估计

8.2 进度计划

8.2.1 甘特图

甘特图 (Gantt Chart) 用横条表示各活动的起止时间，直观展示进度与并行关系，是沟通与跟踪的主力工具。下面是本章示例项目的甘特图（时间单位统一，关键活动以深色标出）：

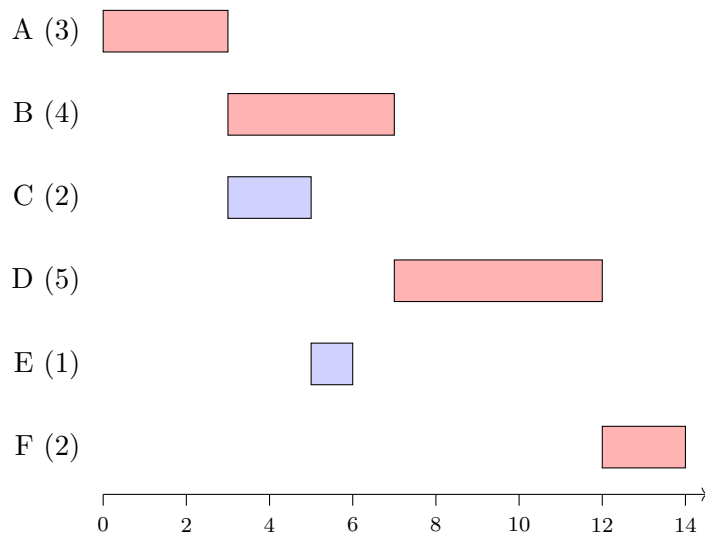


图 8.1: 项目甘特图（深色为关键活动，括号内为工期）

8.2.2 PERT 与关键路径法 CPM

关键路径法 (CPM) 把项目建成活动在网络 (AON)：节点是活动，箭头表示先后约束。对每个活动计算四个时间：

- 最早开始 ES 、最早完成 $EF = ES + \text{工期}$ ：沿网络正向推进， $ES = \max(\text{所有前置的 } EF)$ 。
- 最晚完成 LF 、最晚开始 $LS = LF - \text{工期}$ ：沿网络反向推进， $LF = \min(\text{所有后继的 } LS)$ ，终点 LF 取项目总工期。

总浮动时间 (Total Float) 为

$$TF = LS - ES = LF - EF \quad (8.5)$$

$TF = 0$ 的活动组成关键路径，其长度等于项目最短工期；关键活动一旦延误，整个项目立即延误。示例项目活动与计算结果如下：

表 8.4: 示例项目活动表与 CPM 计算结果							
活动	工期	前置	ES	EF	LS	LF	TF
A	3	—	0	3	0	3	0
B	4	A	3	7	3	7	0
C	2	A	3	5	9	11	6
D	5	B	7	12	7	12	0
E	1	C	5	6	11	12	6
F	2	D, E	12	14	12	14	0

关键路径为 $A \rightarrow B \rightarrow D \rightarrow F$ ，工期 $3 + 4 + 5 + 2 = 14$ 。活动 C、E 各有 6 个时间单位的浮动，可在不影响总工期的前提下适当推迟或调配资源。

8.3 风险管理

8.3.1 风险识别与评估

风险是「尚未发生、但一旦发生会危害项目」的潜在事件。管理流程为：识别 → 分析（概率 × 影响）→ 制定应对 → 监控。风险值用「发生概率」与「影响程度」的乘积度量：

$$R = P(\text{风险}) \times I(\text{影响}) \tag{8.6}$$

常见风险来源分四类：**技术**（技术不成熟、需求不稳）、**人员**（流失、技能不足）、**组织**（预算削减、变更失控）、**外部**（供应商、法规）。下表给出典型风险登记册：

表 8.5: 风险登记册示例（概率、影响取 1 ~ 5）				
风险	概率 P	影响 I	$R = P \times I$	应对策略
需求蔓延	4	4	16	缓解：变更控制、原型确认
关键技术不成熟	3	4	12	规避：技术预研、留备选
核心人员流失	2	5	10	缓解：文档化、结对、备份
第三方组件延误	3	3	9	转移：合同条款、多供应商
预算削减	3	5	15	缓解/接受：优先级排序与储备

8.3.2 风险应对策略

针对高 R 值风险，需选择并落实应对策略：

- 规避 (Avoid)：改变计划以消除风险源，如采用成熟技术替代未验证方案。
- 转移 (Transfer)：把后果转给第三方，如外包、保险、固定价合同。
- 缓解 (Mitigate)：降低发生概率或影响，如增加评审、冗余设计、缓冲。
- 接受 (Accept)：风险影响小或应对成本过高时，预留应急储备并监控。

表 8.6: 概率-影响矩阵与处置建议

	影响低	影响中	影响高
概率高	缓解	缓解/规避	规避/转移
概率中	接受	缓解	规避/转移
概率低	接受	接受	缓解/转移

风险管理的输出是可跟踪的**风险登记册与应急储备**：不仅登记风险，还要指定负责人、触发条件与应对动作，并随项目推进持续复审。

8.4 软件质量

8.4.1 ISO 质量模型

ISO/IEC 9126（后演化为 ISO/IEC 25010）从多个质量特性刻画软件质量，常归纳为「功能性、可靠性、易用性、效率、可维护性、可移植性」等：

表 8.7: ISO 质量特性

质量特性	含义
功能性 (Functionality)	满足所声明与隐含需求的能力
可靠性 (Reliability)	在规定条件下维持规定性能的能力
易用性 (Usability)	被理解、学习、使用的难易程度
效率 (Efficiency)	时间与资源利用的效率
可维护性 (Maintainability)	被修改、纠错、适应、完善的能力
可移植性 (Portability)	迁移到其他环境的能力

8.4.2 质量度量

质量特性需用可度量的指标落地。常用度量包括：

$$D = \frac{\text{缺陷总数}}{\text{KLOC}} \quad (\text{缺陷密度, defects/KLOC}) \tag{8.7}$$

$$MTBF = MTTF + MTTR \tag{8.8}$$

$$A = \frac{MTTF}{MTTF + MTTR} \quad (\text{可用度, Availability}) \tag{8.9}$$

其中 *MTTF* 为平均无故障时间，*MTTR* 为平均修复时间。由公式可见：提高可用度有两条路——降低故障率（增大 *MTTF*）或加快恢复（减小 *MTTR*）。工程上常在两者间权衡，例如高可用系统用冗余提高 *MTTF*、用自动故障转移压缩 *MTTR*。

表 8.8: 常用质量度量指标

指标	定义	关注点
缺陷密度	缺陷数 / KLOC	开发与测试阶段的缺陷控制
可靠性	失效概率、 <i>MTTF</i>	长时间稳定运行能力
<i>MTBF</i>	<i>MTTF</i> + <i>MTTR</i>	平均故障间隔，可维护系统
可用度 <i>A</i>	<i>MTTF</i> / <i>MTBF</i>	服务可用的时间占比

度量本身不是目的：质量模型提供「看哪些维度」，度量提供「量多大」，而评审、测试与过程改进才是「把质量做出来」的手段。